



VESTORA

ملف مذاكرة · مادة تحضير شخصية للمناقشة

Database

الميكانيكا الكاملة: AppDbContext، الكيانات، العلاقات، الفهارس، الحذف الناعم، وستة وعشرون ترحيلًا على قاعدة مشتركة

هذا الملف للمذاكرة فقط. ليس تقريرًا ولا وثيقة تُسلّم — الغرض منه أن تفهم جزءك من المشروع فهمًا كاملاً وتقدر تشرحه وتدافع عنه.

المحتويات

1. Your Responsibility, Exactly	4
1.1. The One Sentence That Sums Up Your Role	4
1.2. The Safe Line Between You and the Leader	4
1.3. What You Own, in Detail	5
1.4. What's Not Yours (Leave It to Its Owner)	5
1.5. Vocabulary You Must Get Right	5
1.6. Why Database Mechanics Deserves Its Own Member	5
2. The Big Picture: EF Core, Code-First, and How a Class Becomes a Table	6
2.1. What an ORM Actually Is	6
2.2. Code-First: The Code Is the Source, Not the Database	7
2.3. How a Class Becomes a Table: Conventions	7
2.4. Three Ways to Override Conventions	8
2.5. The <code>DbContext</code> : What It Actually Is	9
2.6. No Repository Layer — and Why	10
2.7. The Difference Between <code>DbSet</code> and the Table	10
3. Entity and Relationship Map	11
3.1. The Full Picture	11
3.2. Relationships and Delete Behavior	12
3.3. The Full Money Trail	13
3.4. Ownership: Who Owns What	14
4. Entities, Column by Column	15
4.1. <code>Investment</code> — The Relationship Itself	15
4.2. <code>Project</code> — The Venture	15
4.3. <code>User</code> — The User	16
4.4. <code>Status</code> and <code>Stage</code> : Two Different Axes	17
4.5. <code>FundingRequest</code> — The Money Request	18
4.6. <code>PaymentTransaction</code> — The Payment Attempt	18
4.7. <code>PaymentEvent</code> — The Provider's Confirmation	19
5. Concurrency: When Two Writes Happen at Once	19
5.1. How It Works	20
6. Queries: From LINQ to SQL	20
6.1. Deferred Execution	21
6.2. Tracking and No-Tracking	21
6.3. The N+1 Problem	21
6.4. Translation Isn't Always Possible	21
7. Inheritance: Three User Types, One Table	22
7.1. What Actually Happens	22
7.2. Why TPH, Not a Table per Type	22
8. Soft Delete: The Row Stays, It Just Disappears	23
8.1. The Problem	23
8.2. The Fix: A Column Plus a Global Filter	23
8.3. How to Bypass the Filter When You Need To	24
9. Indexes: Speed, and Something More Important Than Speed	24

9.1. Type One: Performance Indexes	24
9.2. Type Two: Indexes That Enforce Rules	25
10. Two Rules That Never Break	26
10.1. Rule One: Every Date Is UTC	26
10.2. Rule Two: Every Money Column Is <code>HasPrecision(18, 2)</code>	26
11. Migrations: The Schema's History	27
11.1. The Idea	27
11.2. The Mechanism Inside: Three Things You Must Know	28
11.3. Why Conflicts Happen	28
11.4. The Safest Command You Have	28
11.5. All Twenty-Six Migrations	28
12. A Shared Database: The Risk and the Discipline	30
12.1. The Problem	30
12.2. Checklist Before You Apply	30
12.3. The Real Example: <code>AddUserOnboardedAt</code>	31
13. File Storage: Why the Bytes Live in the Database	31
13.1. The Current Setup	31
13.2. The Alternative That Wasn't Used	32
14. Known Limitations in Your Area	32
15. Explaining the ERD in 5 Minutes	33
15.1. Minute One: The Big Picture	33
15.2. Minute Two: Why <code>Investment</code> Isn't a Join Table	33
15.3. Minute Three: The Money Chain	34
15.4. Minute Four: The Part That Matters	34
15.5. Minute Five: The Close	34
16. Expected Questions and Their Answers	35
17. Suggested Study Plan	37
18. Quick Review Before the Defense	38
18.1. The Numbers to Memorize	38
18.2. Where to Find Everything	39
18.3. The First Sentence You Say If Asked to Explain	39
18.4. One Last Thing	39

1. Your Responsibility, Exactly

قبل ما تفتح سطر كود واحد، لازم تعرف حدود جزءك. الحدود دي مش شكلية — هي اللي هتخليك في المناقشة تجاوب بثقة على اللي تخصك، وتحول بأمانة اللي مش بتاعك للزميل اللي بيملكه. الاتنين علامة إنك فاهم، مش علامة ضعف.

1.1. The One Sentence That Sums Up Your Role

اعرف ده كويس

إنت مسؤول عن **ميكانيكا** طبقة البيانات في Vestora: إيه الجداول اللي موجودة فعلاً، إزاي الكلاس بيتحول لجدول، إزاي العلاقات متظبطة في `AppDbContext.cs`، إيه الفهارس ولمين بتخدم، إزاي الحذف الناعم شغال، إزاي التواريخ والأرقام العشرية متحكوم فيها، وإزاي الترحيلات (migrations) بتتولد وتراجع وتطبق بأمان على قاعدة بيانات **مشتركة وبعيدة**.

يعني لو حد سألك «فين بالظبط مكتوب إن مشروع واحد ممكن يكون فيه أكثر من صورة؟» لازم تفتح `AppDbContext.cs` وتوريه السطر. ولو سألك «إيه اللي بيمنع إن نفس طلب التمويل يتدفع مرتين؟» لازم تقول اسم الفهرس بالحرف: `UX_PaymentTransactions_OneSucceededPerRequest`، وتشرح إن ده قيد على مستوى قاعدة البيانات مش شرط في كود التطبيق.

1.2. The Safe Line Between You and the Leader

ده الجزء اللي لازم تقوله بصوت عالي في المناقشة لو اتسألت:

خد بالك

إنت **ما بتملكش** التبرير التصميمي عالي المستوى للسكينا. سؤال زي «ليه أصلاً فصلتوا `Investment.Status` عن `Investment.Stage`؟» أو «ليه `FundingRequest` كيان مستقل مش عمودين على `Investment`؟» — ده سؤال **قرار معماري**، وصاحبه هو قائد الفريق. اللي بتملكه إنت: إن الفصل ده **موجود فعلاً** في الكود، والعمودين اتعملوا في أي ترحيل، وكل واحد فيهم نوعه إيه وطوله كام، وإيه الفهرس اللي بيخدم كل واحد فيهم.

الفرق عملي جداً. خد المثال ده:

مثال

سؤال على الحد الفاصل

اللجنة: ليه بتحتفظوا بالصف بتاع العلاقة لما المؤسس يرفض المستثمر، بدل ما تمسحوه؟

الإجابة الصح منك: القرار ده قرار تصميمي بيملكه قائد الفريق، وهو اللي يشرح المنطق التجاري وراه. اللي أقدر أثبتة إنا: القاعدة مكتوبة في `AGENTS.md` كقاعدة غير قابلة للكسر، والتنفيذ الفعلي بتاعها إن `Investment` مافيهش أي `OnDelete` بيمسح الصف عند الرفض — الرفض بيغير `Status` و `Stage` لقيمة `"Declined"` وبس. وعشان كده الفهرس الفريد `UX_Investments_OneLivePerInvestor` مفلتر على `[Status] IN ('Pending', 'Approved')` تحديداً — علشان الصف المرفوض يخرج بره الفلتر ويسبب المستثمر حر يقدم تاني.

لاحظ: ما ادعيتش إنك صاحب القرار، لكن أثبتت إنك عارف مكانه في الكود بالحرف. ده أقوى بكثير من إجابة عامة.

1.3. What You Own, in Detail

الملف	إيه اللي فيه
Data/AppDbContext.cs	قلب شغلك كله. كل DbSet، كل علاقة، كل فهرس، كل HasPrecision، ال TPH discriminator، وفلاتر الحذف الناعم.
Data/Models/*.cs	٣٥ كلاس كيان. الأعمدة، أنواعها، ال Data Annotations، وال navigation properties.
Migrations/*.cs	٣٦ ترحيلًا. تاريخ السكيميا من أول جدول لآخر فهرس.
Migrations/AppDbContextModelSnapshot.cs	صورة الموديل الحالية اللي ال EF Core بيقارن عليها لما تولّد ترحيل جديد.
Services/UtcDateTimeConverter.cs	آلية فرض إن كل تاريخ يخرج على السلك بعلامة UTC.
.config/dotnet-tools.json	تعريف أداة dotnet-ef كأداة محلية للمشروع بإصدار مثبت.

جدول 1 ملفاتك في المشروع.

1.4. What's Not Yours (Leave It to Its Owner)

الموضوع	صاحبه
ليه السكيميا شكلها كده؟ منطق المنتج ورا كل كيان	قائد الفريق
حساب أرقام التمويل: FundingMath.cs	عضو ال Backend / المنطق
دورة حياة الدفع في PaymentService.cs	عضو الدفع/التكامل
ال API endpoints والتحقق من الملكية	عضو ال Backend
الواجهة، الترجمة، ال RTL	عضو ال UI/UX

بس خد بالك من نقطة مهمة: مجرد إن حاجة مش بتاعتك ما يعنيش إنك تجهل وجودها. لازم تعرف إن FundingMath.cs موجود وإنه بيقرا من الجداول بتاعتك بأي فلاتر — لأن دي هي اللي بتفسر ليه الفهارس اللي إنت عاملها على Investment(ProjectId, Status) و PaymentTransaction(ProjectId, Status) موجودة.

1.5. Vocabulary You Must Get Right

في Vestora مفردات الفلوس محكومة بقواعد صارمة مكتوبة في AGENTS.md. لو نطقها غلط في المناقشة، ده بيقرا إنك مش فاهم المنتج:

اعرف ده كويس
- Interest ≥ Committed ≥ Funded. الثلاثة أرقام مختلفة، مش مرادفات. - موافقة المؤسس على مستثمر = committed (التزام). ما تقولش عليها «raised» ولا «تم جمعه». - Funded (تمويل فعلي) = صف PaymentTransaction حالته Succeeded وبس. لا أقل. - المنصة بتقول «reviewed» (تمت مراجعته) — ما بتقولش «vetted» ولا «verified» ولا «موثّق». لأن المنصة ما تقدرش تثبت ده.

1.6. Why Database Mechanics Deserves Its Own Member

ممكن حد يستهون بالدور ويقول «يعني إيه؟ جداول؟». الرد العملي:

1. سلامة الفلوس مش في كود **C#**، هي في الفهارس. أربع قيود فريدة مفلترة في المشروع بتمنع دفعتين لنفس الطلب، وطلبين مفتوحين لنفس العلاقة، وويبهوك يتطبّق مرتين. الكود ممكن يخسر السباق (race)، الفهرس لأ.
2. الحذف الناعم غلطة فيه بتسرّب بيانات. لو فلتر واحد اتشال، حسابات محذوفة هتظهر في نتائج البحث.
3. الترحيل الغلط على قاعدة مشتركة بيوقف الفريق كله. ٦ أفراد على نفس قاعدة البيانات البعيدة.
4. الدقة العشرية. `decimal` من غير `HasPrecision` يبقى `decimal(18,0)` في SQL Server — يعني كل الكسور بتتقصّ بصمت. ده مش bug ظاهر، ده فلوس بتختفي.

خلاصة

إنت بتملك الميكانيكا: `AppDbContext.cs`، ٢٥ كيان، ٢٦ ترحيل، الفهارس، الحذف الناعم، قاعدة UTC، وقاعدة `HasPrecision(18, 2)`. مش بتملك التبرير التصميمي عالي المستوى — ده للقائد. اعرف الحد، وقوله بوضوح لو اتسألت.

اختبر نفسك

1. اذكر ثلاث حاجات إنت مسؤول عنهم بالاسم من الكود.
`AppDbContext.cs` وكل ما فيه من علاقات وفهارس، مجلد `Migrations/` بال ٢٦ ترحيلًا، و `UtcDateTimeConverter.cs`.
2. لو اتسألت «ليه فصلتوا Status عن Stage؟» تعمل إيه؟
أقول إن المنطق التصميمي بيملكه قائد الفريق، وأعرض اللي أقدر أثبته: العمودين موجودين على `Investment`، `Stage` اتضاف في ترحيل `Phase2_InvestmentPipeline` بنوع `nvarchar(20)` وقيمة افتراضية "New"، و `Status` اتحوّل في نفس الترحيل لـ `nvarchar(450)` علشان يتفهرس.
3. ما الفرق بين `Committed` و `Funded` في مفردات المشروع؟
`Committed` = المؤسس وافق على العلاقة — التزام معلن، مش فلوس. `Funded` = فيه `PaymentTransaction` حالته `Succeeded`. ما ينفعش تقول «raised» على الأولانية.
4. ليه الفهرس أقوى من شرط في كود **C#** لمنع الدفع المزدوج؟
شرط الكود هو قراءة بعدها كتابة؛ طلبين متزامنين ممكن الاتنين يقرأوا «مفیش» ويكتبوا. الفهرس الفريد بيتحقق وقت الإدخال نفسه على مستوى المحرك، فواحد بس هينجح.

2. The Big Picture: EF Core, Code-First, and How a Class Becomes a Table

الفصل ده أساس كل اللي بعده. لو الجزء ده مش راسخ، باقي التفاصيل هتفضل حفظ.

2.1. What an ORM Actually Is

في التطبيق إنت شغّال بكائنات **C#**: `User`، `Investment`، `Project`. في قاعدة البيانات فيه صفوف وأعمدة. الـ ORM (اختصار Object-Relational Mapper) هو الطبقة اللي بتترجم بين العالمين: بتاخذ كائن وتحوّله لصف، وتاخذ صف وترجّعه كائن، وتاخذ استعلام LINQ وتحوّله لـ SQL. Entity Framework Core (أو EF Core) هو الـ ORM الرسمي لـ .NET، والمشروع مستخدم منه إصدار متوافق مع .NET 8، مع مزوّد SQL Server.

ملحوظة

الترجمة مش سحر. لما تكتب `context.Projects.Where(p => p.OwnerId == 5)` ، EF Core بيبنى شجرة تعبير (Expression Tree) ويحولها `SELECT ... FROM Projects WHERE OwnerId = 5`. لو كتبت حاجة EF Core مش عارف يترجمها، بيرمي استثناء أو — الأسوأ — بيحب الصفوف كلها للذاكرة ويرشّح فيها هناك. علشان كده `FundingMath.cs` مكتوب كله بـ `Expression<Func<...>>`: علشان يفضل مترجم لـ SQL.

2.2. Code-First: The Code Is the Source, Not the Database

فيه أسلوبين للشغل مع EF Core:

الأسلوب	يعني إيه
Database-First	إنت بتعمل الجداول بـ SQL الأول، وبعدين تولّد كلاسات <code>C#</code> منها.
Code-First	إنت بتكتب كلاسات <code>C#</code> ، و EF Core بيولّد الجداول منها عن طريق الترحيلات.

اعرف ده كويس

Vestora شغال **Code-First**. مصدر الحقيقة هو كلاسات `Data/Models/` + إعدادات `AppDbContext.OnModelCreating`. أي تعديل في السكيمات بيدأ من الكود، وبعدين يتولّد ترحيل. ما بنفتحش SSMS ونعمل `ALTER TABLE` بإيدنا — لأن كده الموديل في الكود والقاعدة يبقوا مختلفين، و EF Core مش هيعرف.

2.3. How a Class Becomes a Table: Conventions

EF Core بيشتغل باتفاقيات ضمنية. من غير ما تكتب أي إعداد، هو بيقترض:

1. **اسم الجدول** = اسم الـ `DbSet` في الـ `context`. عندنا `public DbSet<Project> Projects` ← الجدول اسمه `Projects` (جمع).
2. **المفتاح الأساسي** = الخاصية اللي اسمها `Id` أو `<TypeName>Id`. في `Project` عندنا `public int Id` → صار `PK_Projects`.
3. `int Id` بيتعمله `IDENTITY(1,1)` تلقائيًا — يعني ترقيم تلقائي.
4. **المفتاح الأجنبي** = خاصية اسمها `<Navigation>Id`. في `Project` فيه `public int OwnerId` مع `public Innovator Owner` → EF بيربطهم لوحده.
5. **القابلية للـ null**: `string?` ← عمود `NULL`. `string` ← عمود `NOT NULL`.
6. `string` من غير طول ← `nvarchar(max)`.

مثال

قراءة كلاس بسيط وتوقع جدول

```

1 public class ProjectImage
2 {
3     public int Id { get; set; }
4     public byte[] ImageData { get; set; }
5     public string ContentType { get; set; }
6     public DateTime UploadedAt { get; set; }
7     public int ProjectId { get; set; }
8     public Project Project { get; set; }
9 }

```

اللي هيطلع من ده من غير أي إعداد إضافي:

```

1 CREATE TABLE [ProjectImages] (
2     [Id] int NOT NULL IDENTITY(1,1),
3     [ImageData] varbinary(max) NOT NULL,
4     [ContentType] nvarchar(max) NOT NULL,
5     [UploadedAt] datetime2 NOT NULL,
6     [ProjectId] int NOT NULL,
7     CONSTRAINT [PK_ProjectImages] PRIMARY KEY ([Id]),
8     CONSTRAINT [FK_ProjectImages_Projects_ProjectId]
9         FOREIGN KEY ([ProjectId]) REFERENCES [Projects]([Id])
10 );
11 CREATE INDEX [IX_ProjectImages_ProjectId] ON [ProjectImages]([ProjectId]);

```

لاحظ حاجتين: `byte[]` بقى `varbinary(max)` ، و EF Core عمل فهرس تلقائي على المفتاح الأجنبي. الفهرس ده مش إنت اللي كتبته — دي اتفاقية.

2.4. Three Ways to Override Conventions

لما الاتفاقية ما تكفيش، عندك ثلاث مستويات:

2.4.1. 1. Data Annotations (Attributes on the Property)

بتتخط جوه كلاس الكيان نفسه. المشروع مستخدمها كثير:

```

1 [Required]
2 [StringLength(100)]
3 public string UserName { get; set; }
4
5 [EmailAddress]
6 [StringLength(255)]
7 public string Email { get; set; }
8
9 [Timestamp]
10 public byte[]? RowVersion { get; set; }

```

`[StringLength(100)]` بيخلي العمود `nvarchar(100)` بدل `nvarchar(max)`. `[Timestamp]` بيخلي العمود من نوع `rowversion` — وده اللي بيدّي للـ `PaymentTransaction` حمايتها من التعديل المتزامن.

خد بالك

خد بالك: `[Required]` و `[EmailAddress]` ليهم دورين مختلفين. `[Required]` بيأثر على السكيمات (`NOT NULL`) و `[EmailAddress]` على التحقق في ASP.NET. `[EmailAddress]` تحقق بس — ما بيعملش أي حاجة في قاعدة البيانات. ما تقولش في المناقشة إن الإيميل «متحقق منه في القاعدة» لأن ده غلط.

2.4.2. 2. Fluent API in `OnModelCreating`

ده المكان الأساسي بتاعك. كل حاجة ما ينفعش تتقال بـ annotation — العلاقات المرغبة، الفهارس المرغبة، الفهارس المغلطة، سلوك الحذف، الدقة العشرية، الحذف الناعم — بتكتب هنا:

```
1 modelBuilder.Entity<Investment>()
2     .Property(i => i.Amount)
3     .HasPrecision(18, 2)
4     .IsRequired();
```

اعرف ده كويس

ال Fluent API أقوى من ال annotations. لو الاتنين اتعارضوا على نفس الحاجة، ال Fluent API بيكسب. وعلشان كده كل القواعد اللي ما ينفعش تتكسر في Vestora مكتوبة Fluent — لأن مكانها مركزي وواضح في ملف واحد.

2.4.3. 3. Custom Conventions

المشروع ما بيستخدمش المستوى ده. لو اتسألت «ليه ما عملتوش convention تحط `HasPrecision` على كل `decimal` تلقائيًا؟» — الرد الأمين: مش معمول في المشروع، والقاعدة بتتفرض حاليًا بالمراجعة اليدوية وبالقائمة اللي في `AGENTS.md`.

2.5. The `DbContext`: What It Actually Is

`AppDbContext` بيرث من `DbContext`، وهو بيلعب ثلاث أدوار في نفس الوقت:

1. **وصف الموديل** — كل `DbSet<T>` يقول لـ EF Core «الكيان ده جزء من الموديل»، و `OnModelCreating` بيضبط تفاصيله.
2. **وحدة عمل (Unit of Work)** — بيتتبع كل التغييرات اللي حصلت على الكائنات، وينفذها كلها مرة واحدة في `SaveChanges()` جوه معاملة (transaction) واحدة.
3. **مصنع استعلامات** — `context.Projects` هو نقطة بداية أي LINQ query.

```
1 public class AppDbContext : DbContext
2 {
3     public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }
4
5     public DbSet<User> Users { get; set; }
6     public DbSet<Investor> Investors { get; set; }
7     public DbSet<Innovator> Innovators { get; set; }
8     public DbSet<Project> Projects { get; set; }
9     public DbSet<Investment> Investments { get; set; }
10    // باقي الـ DbSet
11 }
```

ال constructor بياخد `DbContextOptions<AppDbContext>` — يعني الإعدادات (سلسلة الاتصال والمزود) بتتحقن من بره، مش مكتوبة جوه الكلاس. الحقن ده بيحصل في `Program.cs`:

```
1 builder.Services.AddDbContext<AppDbContext>(options =>
2     options.UseSqlServer(selectedConnectionString));
```

ملحوظة

`AddDbContext` سيسجل ال context بعمر **Scoped** — يعني نسخة واحدة لكل طلب HTTP. ده مقصود: ال `DbContext` مش thread-safe ومش المفروض يعيش طويل. المتحكم (controller) بياخد النسخة دي في ال constructor بتاعه ويشغل بيها.

2.6. No Repository Layer — and Why

في مشاريع كتير بتلاقى `IProjectRepository` وطبقة تانية فوق `DbContext`. Vestora ما بيعملش كده، والقاعدة دي مكتوبة صراحةً في `AGENTS.md`:

اعرف ده كويس

المتحكمات (controllers) بتكلم `AppDbContext` مباشرةً. مفيش repositories. الخدمات (`Services/`) موجودة للمصادقة والدفع والحسابات بس، مش كطبقة وصول بيانات. ده يعني إن ال `DbContext` بتاعك مش تفصلة داخلية — هو الواجهة اللي كل ال backend بيشتغل عليها. أي فلتر عام إنت بتحطه، بيسري على كل المشروع بلا استثناء.

لو اتسألت «ليه؟» — ده قرار معماري بيملكه القائد. اللي إنت بتملكه: إنك تعرف إن ده الوضع، وتعرف نتيجته المباشرة على شغلك — إن `HasQueryFilter` عندك بيطبق فعلياً في كل مكان من غير ما حد يفتكر يكتبه.

2.7. The Difference Between `DbSet` and the Table

خد بالك

فيه خمس جداول في قاعدة البيانات **مالهمش** `DbSet` مباشر في `AppDbContext`، وبرضه EF Core عارفهم. إزاي؟ لأن EF Core بيكتشف الكيانات من ال navigation properties كمان، مش من ال `DbSet` بس. مثال: `Reply` عنده `DbSet`، لكن لو ما كانش عنده، EF Core كان هيكتشفه من `Comment.Replies`. والعكس: `Admin` **مالوش** `DbSet` خالص — هو معروف من خلال `HasDiscriminator().HasValue<Admin>("Admin")` بس.

في نفس الوقت، فيه جدولين اتعملوا في الترحيل الأولاني واتشالوا بعدين خالص: `Payments` و `ProjectAnalyses`. اتمسحوا في ترحيل `ExpansionRelationshipWorkspace` لما نظام الدفع الحقيقي (`PaymentTransaction`) حلّ محلهم. ده تفصلة مهمة عن تاريخ السكيما هتحتاجها في فصل Migrations.

خلاصة

- EF Core = مترجم بين كائنات `C#` وصفوف SQL.
- Vestora شغل **Code-First**: الكلاس هو الأصل، والجدول نتيجة.
- EF بيشتغل باتفاقيات (اسم الجدول، `Id` كمفتاح، `<Nav>Id` كمفتاح أجنبي، فهرس تلقائي على كل FK).
- تغيير الاتفاقية بـ annotations أو بال Fluent API. Fluent أقوى وهو مكان كل قواعد المشروع الصارمة.
- `AppDbContext` = وصف موديل + وحدة عمل + مصنع استعلامات، وعمره Scoped لكل طلب.
- مفيش repositories: المتحكمات بتكلم ال context مباشرة، فأني فلتر عام بيسري على كل المشروع.

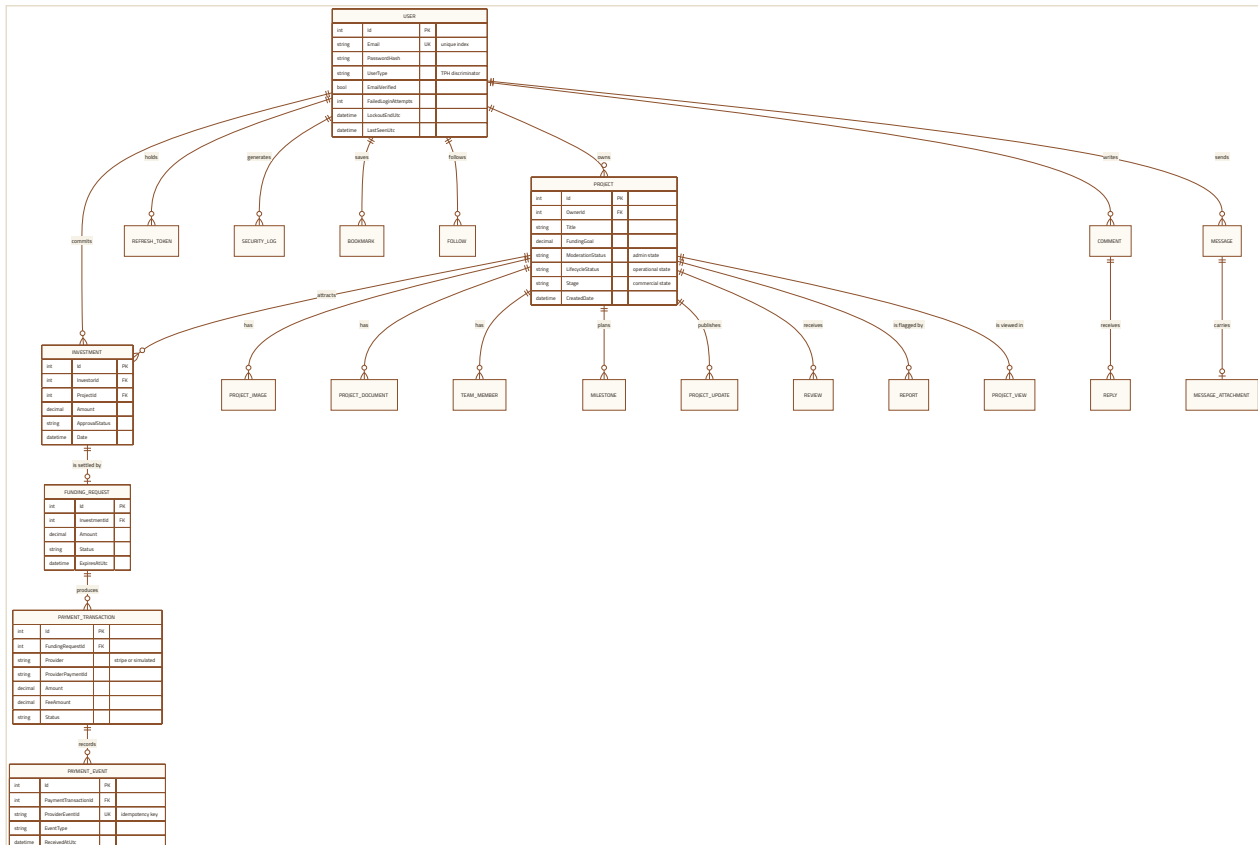
اختبر نفسك

1. كلاس فيه `public string? Note { get; set; }` من غير أي سمة — العمود هيطلع إيه؟
`Note nvarchar(max) NULL` ال `?` بيخليه nullable، وغياب `StringLength(max)` بيخليه `max`.
2. EF Core بيعمل فهرس تلقائي على إيه من غير ما تطلب؟
على كل مفتاح أجنبي (FK). مثلاً `IX_ProjectImages_ProjectId` اتعمل لوحده.
3. لو حطيت `[StringLength(50)]` وكمان `HasMaxLength(100)` في ال Fluent API، الناتج كام؟
١٠٠. ال Fluent API بيكسب على ال annotations.
4. ليه غياب طبقة ال repository مهم بالذات لشغلك إنت؟
لأن المتحكمات بتستخدم ال `DbContext` مباشرة، فال global query filters اللي إنت كاتبها بتسري على كل استعمال في المشروع من غير ما حد يفتكر يطبقها.
5. كيان `Admin` مالوش `DbSet` — إزاي EF Core عارفه؟
من إعداد ال TPH: `HasDiscriminator<string>("UserType").HasValue<Admin>("Admin")` في `OnModelCreating`.

3. Entity and Relationship Map

ده الفصل اللي المفروض تعرفه أكثر من أي حد في الفريق. لو اللجنة فتحت ال ERD وقالت «اشرح»، الكلام اللي جاي هو اللي هتقوله.

3.1. The Full Picture



شكل 1 المخطط الأساسي: المستخدم، المشروع، والعلاقة بينهم. لاحظ إن `Investment` مش مجرد جدول ربط — عنده حالة ومرحلة وتاريخ.

3.2. Relationships and Delete Behavior

أهم حاجة في أي علاقة عندنا مش بس مين مربوط بمين، لكن إيه اللي بيحصل لما الطرف الأساسي يتمسح. ده اسمه **OnDelete** أو delete behavior، وكل اختيار فيه في المشروع اتحدد بقصد.

العلاقة	السلوك	ليه كده
Project ← Innovator	Cascade	المشروع مالوش معنى من غير صاحبه. لو المؤسس اتشال، مشاريعه تروح معاه.
Investment ← Investor	Restrict	الالتزام سجل مالي. ممنوع يختفي عشان المستثمر اتشال — لازم حد ياخذ قرار صريح الأول.
Message ← Receiver / Sender	Restrict	محادثة بين طرفين. لو مسحنا رسائل طرف، تاريخ الطرف التاني بيتخرم.
MessageAttachment ← Message	Cascade	المرفق تبع الرسالة نفسها (علاقة 1:1 بنفس المفتاح). الرسالة تروح، البايتس تروح.
RefreshToken ← User	Cascade	توكن جلسة. مالوش قيمة من غير صاحبه.
SecurityLog ← User	SetNull	عكس اللي فوق: السجل الأمني لازم يفضل حتى لو الحساب راح — بيبقى مجهول الهوية بدل ما يتمسح.
FundingRequest ← Investment	Cascade	الطلب جزء من العلاقة نفسها.
FundingRequest ← TermSheet	NoAction	سجل مالي تاني. ممنوع يختفي لأن الشروط اللي وراه اتشالت.
PaymentTransaction ← FundingRequest	Cascade	المحاولة تبع الطلب.

جدول 2 كل سلوك حذف في المشروع والسبب وراه.

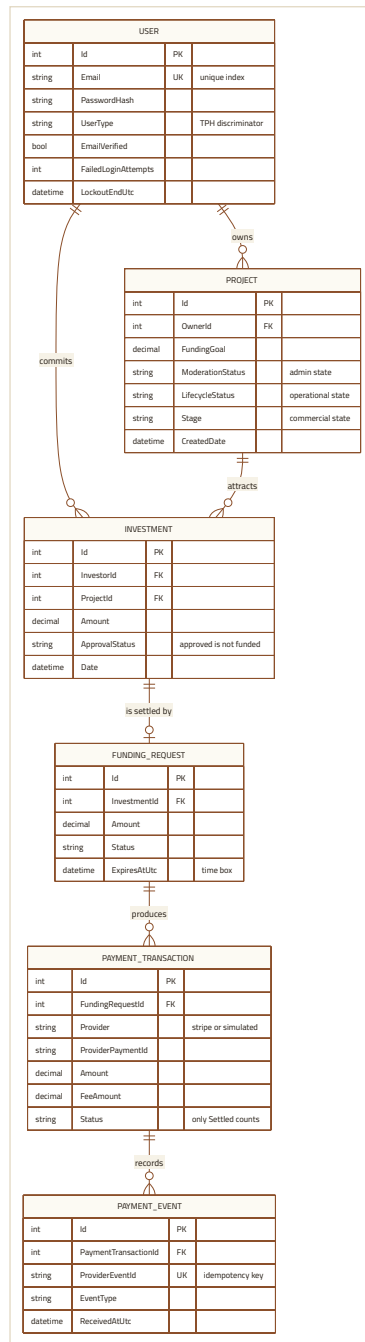
اعرف ده كويس

القاعدة اللي بتفسر الجدول ده كله: أي حاجة فيها فلوس أو أي حاجة دليل — متمسحش تلقائيًا. عشان كده **Investment** و **FundingRequest** ← **TermSheet** بـ **Restrict** و **NoAction**، بينما **MessageAttachment** و **RefreshToken** بـ **Cascade** عادي.

خد بالك

SecurityLog هو الاستثناء الوحيد بـ **SetNull** — ودي نقطة ممتازة لو اتسألت «إزاي بتوازنوا بين حق المستخدم في الحذف وبين الاحتفاظ بالأدلة؟». الإجابة: السجل بيفضل، بس بيتقطع عن صاحبه.

3.3. The Full Money Trail



شكل 2 سلسلة التمويل من فوق لتحت: العلاقة تولّد طلب، الطلب يولّد محاولات دفع، وكل تأكيد من المزوّد بيتسجّل كحدث مستقل.

السلسلة بالترتيب: **Investment** ← **TermSheet** ← **FundingRequest** ← **PaymentTransaction** ← **PaymentEvent**.

مثال

ليه `PaymentEvent` جدول لوحده؟ لأن الحدث الجاي من المزود (مثلاً Stripe) لازم يتسجل قبل ما أي رقم يتحرك. لو كان مجرد عمود في `PaymentTransaction`، مكنش فيه مكان تسجل فيه ويهوك اترر أو ويهوك وصل قبل ما المستخدم يرجع من صفحة الدفع.

كونه جدول مستقل + الفهرس الفريد على `(Provider, ProviderEventId)` = نفس الحدث مستحيل يتطبق مرتين.

3.4. Ownership: Who Owns What

فحص الملكية بيعتمد على سلاسل المفاتيح الأجنبية دي. لما الكنترولر يسأل «هل المستخدم ده يحق له يعدّل الحاجة دي؟»، السؤال بيتترجم لتتبع السلسلة لحد `UserId`.

المورد	سلسلة الملكية
مشروع	<code>Project.OwnerId</code> ← <code>Innovator.Id</code> — المؤسس بيملك مشروعه.
التزام	طرفين: <code>Investment.InvestorId</code> (المستثمر) و <code>Investment.Project.OwnerId</code> (المؤسس). اللاتنين ليهم حق الوصول، بصلاحيات مختلفة.
ورقة شروط	<code>TermSheet.InvestmentId</code> ← نفس طرفي العلاقة.
طلب تمويل	<code>FundingRequest.InvestmentId</code> ← نفس الطرفين، بس المؤسس هو اللي بينشئه.
معاملة دفع	<code>PaymentTransaction.InvestorId</code> — المستثمر بيدفع، والمؤسس بيشوف النتيجة.
رسالة	<code>Message.SenderId</code> و <code>Message.ReceiverId</code> — الطرفان بس.
مستند مشروع	<code>ProjectDocument.ProjectId</code> ← <code>Project.OwnerId</code> ، مع مستوى ظهور بيحدّد مين ثاني يشوفه.

جدول 3 سلاسل الملكية اللي عليها بيتبني فحص الصلاحية.

اعرف ده كويس

لاحظ إن `Investment` ليه مالكين، مش مالك واحد. ودي أهم نقطة في الجدول ده: أي فحص صلاحية على علاقة لازم يسأل «هل ده المستثمر بتاعها أو صاحب المشروع بتاعها؟» — وبصلاحيات مختلفة لكل واحد. مثال: المؤسس بس هو اللي يقدر يوافق، والمستثمر بس هو اللي يقدر يدفع.

خد بالك

ده بيوضّح قاعدة رقم ٤ في `AGENTS.md`: **فحص الدور مش فحص ملكية**. إن المستخدم دوره `Innovator` مش معناه إنه صاحب المشروع ده بالذات. لازم الاتنين: الدور و السلسلة.

اختبر نفسك

- ليه `Investment` ← `Investor` ب `Restrict` مش `Cascade`؟
لأن الالتزام سجل مالي مشترك بين طرفين. لو اتمسح تلقائياً مع المستثمر، تاريخ المؤسس بيضيع كمان.
- إيه الفرق بين `Cascade` و `SetNull` عملياً؟
`Cascade` بيمسح الصف التابع. `SetNull` بيسيبه موجود ويفضي المفتاح الأجنبي بس — بيستخدم لما السجل نفسه له قيمة من غير صاحبه.
- `MessageAttachment` مفناحه الأساسي إيه؟
`MessageId` نفسه — علاقة 1:1 بمفتاح مشترك، عشان البايثس ما تتحملش مع كل استعلام محادثة.
- لو مسحنا `TermSheet` هيحصل إيه للـ `FundingRequest` المرتبط بيه؟
مفيش حاجة — العلاقة `NoAction`. قاعدة البيانات هترفض المسح أصلاً طالما فيه طلب مربوط.

4. Entities, Column by Column

الفصل ده مرجع. مش لازم تحفظه كله، بس لازم تعرف تفتحه وتلاقي أي عمود بيتسأل عنه.

4.1. Investment — The Relationship Itself

ده أهم كيان في المشروع بعد `User` و `Project`، ولازم تفهم إنه مش جدول ربط.

العمود	النوع	إيه هو
<code>Id</code>	<code>int</code>	المفتاح الأساسي.
<code>Amount</code>	<code>decimal(18,2)</code>	المبلغ اللي المستثمر عايز يدخل بيه.
<code>Date</code>	<code>datetime2</code>	تاريخ الطلب.
<code>Status</code>	<code>string</code>	بوابة التمويل: <code>Pending</code> / <code>Approved</code> / <code>Declined</code> .
<code>Stage</code>	<code>string</code>	مرحلة العلاقة بين الطرفين. الافتراضي <code>New</code> .
<code>StageUpdatedAt</code>	<code>datetime2?</code>	آخر مرة اتحرّكت فيها المرحلة.
<code>ContactInfo</code>	<code>string?</code>	بيانات تواصل المستثمر، بتتشاف بعد الموافقة.
<code>FounderNote</code>	<code>string?</code>	ملاحظة خاصة بالمؤسس — المستثمر مبيشوفهاش.
<code>InvestorNote</code>	<code>string?</code>	ملاحظة خاصة بالمستثمر — المؤسس مبيشوفهاش.
<code>DeclinedReason</code>	<code>string?</code>	سبب الرفض، لو اترفض.
<code>InvestorId</code>	<code>int?</code>	اختياري — لاحظ علامة الاستفهام.
<code>ProjectId</code>	<code>int</code>	إجباري.

جدول 4 أعمدة `Investment`.

خد بالك

`InvestorId` قابل للـ `NULL` بينما `ProjectId` لأ. ودي بالظبط سبب وجود الشرط `[InvestorId] IS NOT NULL` جوّه فلتر القيد `UX_Investments_OneLivePerInvestor` — لأن الصفوف اللي مالهاش مستثمر مينفعش تتزاحم على تفرد.

4.2. Project — The Venture

أكبر كيان في المشروع من ناحية عدد الأعمدة، ومقسّم لخمس مجموعات.

العمود	النوع	إيه هو
<code>Description</code> / <code>Name</code>	<code>string</code>	الأساسيات، إجباريين.
<code>Topic</code> / <code>VideoUrl</code>	<code>string?</code>	اختياريين.
<code>Location</code> / <code>Industry</code> / <code>Category</code>	<code>string?</code>	حقول التصنيف — عليها فهارس التصقّح.
<code>Stage</code>	<code>string?</code>	مرحلة المشروع: <code>Idea</code> / <code>Pre-seed</code> / <code>Seed</code> / <code>Growth</code> . مش مرحلة العلاقة.

العمود	النوع	إيه هو
InvestmentNeeded	decimal(18,2)	هدف الجولة.
Valuation	decimal(18,2)?	التقييم.
EquityOffered	decimal(5,2)?	نسبة الحصّة المعروضة.
UseOfFunds	string?	أوجه استخدام التمويل.
ModerationStatus	string	بوابة النشر. الافتراضي — PendingReview يعني مخفي لحد ما أدمن يراجع.
ModeratedAtUtc / ModerationNote	?	سبب القرار الإداري ووقته.
LifecycleStatus	string	دورة الحياة: Active وغيرها. الافتراضي Active.
RoundClosingNote / RoundOutcome / RoundClosedAtUtc	?	إغلاق الجولة ونتيجتها المعلّنة.
TotalInteractions / InvestorCount	int	عدّادات محسوبة مسبقًا للعرض السريع.
IsDeleted	bool	الحذف الناعم — عليه الفلتر العام.
OwnerId	int	المؤسّس المالك.

جدول 5 أعمدة Project.

خد بالك

ثلاث كلمات «مرحلة» مختلفة في المشروع، متخلّطش بينهم:

- Project.Stage — مرحلة الشركة (فكرة، بذرة، نمو).
- Investment.Stage — مرحلة العلاقة بين مستثمر ومؤسّس.
- Investment.Status — بوابة الفلوس.

ودي غلطة سهلة جدّا تحصل تحت الضغط

اعرف ده كويس

ModerationStatus افتراضي PendingReview — يعني المشروع الجديد مخفي بشكل افتراضي. ده قرار أمان: النشر يحتاج فعل إيجابى من أدمن، مش العكس.

4.3. User — The User

الجدول ده بيضم الثلاث أنواع (TPH)، فأعمدته أكثر من أي جدول تاني.

المجموعة	الأعمدة
الهوية	UserName, Email (فريد), Password (مُجَرَّبًا), UserType (ال (discriminator), UniqueNumber

المجموعة	الأعمدة
الملف الشخصي	<code>TwitterUrl</code> , <code>LinkedinUrl</code> , <code>WebsiteUrl</code> , <code>BriefBio</code> , <code>Phone</code> , <code>BirthDate</code> , <code>CoverImage</code> , <code>ProfileImage</code>
تأكيد الإيميل	<code>EmailVerificationTokenHash</code> , <code>EmailVerifiedAtUtc</code> , <code>IsEmailVerified</code> , <code>EmailVerificationLastSentAtUtc</code> , <code>EmailVerificationTokenExpiresAtUtc</code>
إعادة تعيين كلمة السر	<code>PasswordResetTokenExpiresAtUtc</code> , <code>PasswordResetTokenHash</code> , <code>PasswordResetFailedAttempts</code> , <code>PasswordResetLastRequestedAtUtc</code>
الحماية من التخمين	<code>LastFailedLoginAtUtc</code> , <code>LockoutEndUtc</code> , <code>FailedLoginCount</code>
دورة حياة الحساب	<code>SuspendedAtUtc</code> , <code>IsSuspended</code> , <code>DeletedAtUtc</code> , <code>IsDeleted</code> , <code>LastSeenAt</code> , <code>CreatedAtUtc</code> , <code>SuspensionReason</code>
التفضيلات	<code>OnboardedAtUtc</code> , <code>NotifyOnProjectUpdate</code> , <code>NotifyOnFollow</code>
خاصة بالمستثمر فقط	<code>TicketMax</code> , <code>TicketMin</code> , <code>InvestmentThesis</code> , <code>PreferredIndustries</code> , <code>ListedInDirectory</code> — بتبقى <code>NULL</code> عند المؤسس والأدمن

جدول 6 أعمدة `User` مجمعة بالوظيفة.

اعرف ده كويس

لاحظ نمط متكرر: كل توكن حساس متخزن مُجَزَّأً (`TokenHash` ...) مش خام — توكن تأكيد الإيميل، وتوكن إعادة تعيين كلمة السر، وتوكن التحديث في جدول `RefreshTokens`. لو قاعدة البيانات اتسربت، التوكنات دي مش قابلة للاستخدام. ودي نقطة قوية لو اتسألت عن الأمان على مستوى التخزين.

4.4. Status and Stage : Two Different Axes

دي قاعدة رقم ٣ في `AGENTS.md` ، ومن أكثر الحاجات اللي بتتسأل.

Stage	Status
إحنا فين في العلاقة بين شخصين؟	هل الالتزام ده محسوب في أرقام التمويل؟
تشان مراحل — تحت	<code>Declined</code> · <code>Approved</code> · <code>Pending</code>
تقدّم المحادثة والتفاوض	قرار المؤسس بالموافقة أو الرفض
لأ بشكل مباشر	أيوه — <code>Approved</code> معناها ملتزم

جدول 7 المحوران، ومنعًا للخلط بينهم.

المراحل الثمانية بالنص من `PipelineStages.cs`:

المرحلة	معناها
<code>New</code>	الطلب لسه واصل.
<code>Reviewing</code>	المؤسس بيصّ عليه.
<code>Approved</code>	المؤسس وافق — ودي المرحلة اللي بتتسبب في التمويل.
<code>Contacted</code>	المؤسس تواصل.
<code>InDiscussion</code>	أخذ وردّ شغال.
<code>Committed</code>	الطرفان اتفقوا على الشروط.
<code>Closed</code>	العلاقة اتقفلت — مرحلة نهائية.

المرحلة	معناها
Declined	المؤسس رفض — مرحلة نهائية، والصف يفضل موجود للتاريخ.
Closed	جدول 8 المراحل الثمانية.
Declined	هما المرحلتان النهائيتان (Terminal).

اعرف ده كويس
لو اتسألت «ليه مدمجتوش العمودين في واحد؟» الإجابة: لأنهم بيقيسوا حاجتين مختلفتين. علاقة ممكن تكون Approved في الـ Status (يعني الفلوس محسوبة) وفي نفس الوقت InDiscussion في الـ Stage (يعني لسه بيتكلموا). دمجهم كان هيجلّي مستحيل تعرف رقم التمويل من غير ما تقرا سياق المحادثة.

4.5. FundingRequest — The Money Request

إيه هو	العمود
كود فريد للطلب — عليه فهرس فريد.	Reference
العلاقة اللي الطلب طالع منها.	InvestmentId
المبلغ المطلوب وعملته (الافتراضي USD).	Currency / Amount
Cancelled / Paid / Open وغيرهم.	Status
ورقة الشروط اللي الطلب بيستدعيها (اختياري).	TermSheetId
العرض المضاد من المستثمر.	CounterStatus / CounterAtUtc / CounterNote / CounterAmount
الطلب اللي ده جه بدله، لو فيه.	SupersedesRequestId
ميعاد انتهاء الطلب.	ExpiresAtUtc
عدد التذكيرات اللي اتبعثت.	RemindersSent
تواريخ الإقفال والدفع.	PaidAtUtc / ClosedAtUtc
جدول 9 أعمدة FundingRequest	

4.6. PaymentTransaction — The Payment Attempt

إيه هو	العمود
كود فريد للمعاملة.	Reference
رقم المحاولة — لأن المحاولات بتتعدّد وكل واحدة صف.	AttemptNumber
المبلغ.	Amount
نسبة العمولة بالـ basis points (جزء من عشرة آلاف) — عدد صحيح مش كسر.	FeeRateBps
العمولة المحسوبة والصافي للمؤسس — متجمّدين على الصف.	NetToFounder / FeeAmount
Succeeded / Processing / Initiated وغيرهم.	Status

إيه هو	العمود
هوية العملية عند مزود الدفع.	<code>ProviderPaymentId</code> / <code>ProviderSessionId</code> / <code>Provider</code>
تفاصيل الفشل أو الإلغاء.	<code>CancelReason</code> / <code>FailureMessage</code> / <code>FailureCode</code>
بيانات الاسترجاع الإداري.	<code>RefundedAtUtc</code> / <code>RefundReason</code> / <code>RefundedByAdminId</code>
— طابع التزامن، اقرا الفصل الجاي.	<code>byte[]</code> <code>RowVersion</code>

جدول 10 أعمدة `PaymentTransaction`.

اعرف ده كويس

ليه العمولة **متجمّدة** على الصف بدل ما تتحسب كل مرة؟ لأن نسبة العمولة ممكن تتغير بعد سنة. لو حسبناها لحظة العرض، معاملة قديمة هتظهر برقم مختلف عن اللي اندفع فعلاً. الرقم المتجمّد = الحقيقة التاريخية.

خد بالك

`FeeRateBps` عدد صحيح (`int`) مش `decimal`. الوحدة basis point = ١ من ١٠٠٠٠. يعني ٢٥٠ bps تساوي ٢,٥%. السبب: النسب دي بتتخزن أعداد صحيحة عشان تتجنّب مشاكل التقريب تمامًا.

4.7. `PaymentEvent` — The Provider's Confirmation

إيه هو	العمود
اللاتين مع بعض فهرس فريد — ده قلب منع التكرار.	<code>ProviderEventId</code> + <code>Provider</code>
نوع الحدث الجاي من المزود.	<code>EventType</code>
جه منين: ويهوك ولا رجوع المستخدم من صفحة الدفع.	<code>Source</code>
العمولة الخام زي ما وصلت.	<code>Payload</code>
وقت الاستلام.	<code>ReceivedAtUtc</code>
هل الحدث اتطبق فعلاً ولا اتسجل بس.	<code>Applied</code>
نتيجة التطبيق.	<code>Outcome</code>
مراجعة إدارية للأحداث المشكوك فيها.	<code>ReviewNote</code> / <code>ReviewedByAdminId</code> / <code>ReviewedAtUtc</code>

جدول 11 أعمدة `PaymentEvent`.

اعرف ده كويس

عمود `Applied` مهم: الحدث بيتسجل الأول، وبعدين بيتطبق. لو اتسجل واتطبقش، ده بيان. القاعدة في `AGENTS.md` بتقول بالنص: **سجل الـ `PaymentEvent` قبل تطبيق أي أثر.**

5. Concurrency: When Two Writes Happen at Once

عنده عمود مختلف عن كل اللي فات: `PaymentTransaction`

```
1 public byte[]? RowVersion { get; set; }
```

ده اسمه SQL Server **optimistic concurrency token**. بيغيّر قيمته تلقائيًا مع كل تعديل على الصف.

5.1. How It Works

1. تقرا الصف، وتيجي معاه قيمة **RowVersion** الحالية.
2. تعدّل في الذاكرة.
3. تعمل **SaveChanges()** — و EF Core يضيف في جملة الـ **UPDATE** شرط: «عدّل الصف ده بشرط إن الـ **RowVersion** لسه زي ما قريتها».
4. لو حد ثاني عدّل الصف في الوقت ده، القيمة اتغيّرت، والشرط مبيتحققش، ف صفر صفوف اتعدّلت — و EF Core بيرمي **DbUpdateConcurrencyException**.

مثال

ليه ده مهم في الدفع بالذات

تخيّل ويهوك من Stripe وصل، وفي نفس الجزء من الثانية المستخدم رجع من صفحة الدفع فاتنفذ تحقّق ثاني. الاتنين قروا نفس الصف وحالته **Processing**، والاتنين عايزين يخلّوها **Succeeded**. من غير **RowVersion**: الاتنين بيكتبوا، وممكن أثر التمويل يتطبّق مرتين. مع **RowVersion**: أول واحد بيكسب، والثاني بياخد استثناء ويتعامل معاه — من غير ما رقم واحد في التمويل يتحرّك غلط.

اعرف ده كويس

خد بالك من الفرق: **الفهرس الفريد** بيمنع صقّين متعارضين يتولدوا. **RowVersion** بيمنع تعديلين متزامنين على نفس الصف. الاتنين حمايات مختلفة لمشكلتين مختلفتين، والمشروع مستخدم الاتنين في المسار المالي.

اختبر نفسك

1. **Investment.InvestorId** ليه **nullable**؟
عشان يسمح بصفوف من غير مستثمر مرتبط؛ وعشان كده فلتر القيد الفريد بيشترط **IS NOT NULL**.
2. **علاقة حالتها** **Approved** و**مرحلتها** **InDiscussion** — ده منطقي؟
أيوه تمامًا. **Status** بيقول الفلوس محسوبة، **Stage** بيقول لسه بيتكلموا. محوريين مستقلين.
3. **FeeRateBps** ليه **int**؟
لأنه بالـ basis points (١ من ١٠٠٠٠)، وتخزينه عدد صحيح بيمنع أخطاء التقريب.
4. **ليه العمولة متجمّدة على الصف بدل ما تتحسب وقت العرض؟**
عشان لو النسبة اتغيّرت بعدين، المعاملة القديمة تفضل بالرقم اللي اتدفع فعلاً.
5. **إيه الفرق بين حماية الفهرس الفريد وحماية** **RowVersion**؟
الفهرس بيمنع صفوف متعارضة تتولد؛ **RowVersion** بيمنع تعديلين متزامنين على نفس الصف.

6. Queries: From LINQ to SQL

إنت مش بتكتب SQL في المشروع ده — بتكتب LINQ و EF Core بيترجم. وعشان كده لازم تعرف الترجمة بتحصل إمتى وإزاي، لأن ده أكثر مكان بتحصل فيه مشاكل أداء.

6.1. Deferred Execution

```
1 var q = _context.Projects.Where(p => p.Stage == "Seed"); // لسه مفيش SQL
2 var list = await q.ToListAsync(); // هنا بقى اتنفّذ
```

الاستعلام مبيروحش لقاعدة البيانات لحد ما تطلب النتيجة فعلياً (`ToListAsync` , `FirstOrDefaultAsync` , `CountAsync` ...). ده اسمه **deferred execution**، وفائدته إنك تقدر تركّب شروط على بعض والـ SQL يتولّد مرة واحدة.

6.2. Tracking and No-Tracking

بشكل افتراضي EF Core بيفضل **متتبّع** لكل كيان قراه، عشان يعرف يحفظ التعديلات. لكن لو إنت بتقرا للعرض بس، ده شغل زيادة وذاكرة زيادة:

```
1 await _context.Projects.AsNoTracking().ToListAsync();
```

اعرف ده كويس
القاعدة العملية: **بتقرا عشان تعرض؟ استخدم `AsNoTracking()`. بتقرا عشان تعدّل وتحفظ؟ سيبه متتبّع.**

6.3. The N+1 Problem

أشهر مشكلة أداء في أي ORM:

```
1 var projects = await _context.Projects.ToListAsync();
2 foreach (var p in projects)
3 {
4     var owner = p.Owner; // استعلام لكل مشروع
5 }
```

مية مشروع = مية واحد استعلام. الحل: تجيب المطلوب في استعلام واحد بـ `Include` أو — الأحسن — بـ **إسقاط (projection)** على الأعمدة اللي محتاجها بس:

```
1 await _context.Projects
2     .Select(p => new { p.Id, p.Title, OwnerName = p.Owner.UserName })
3     .ToListAsync();
```

خد بالك
الإسقاط أحسن من `Include` في معظم شاشات العرض، لأن `Include` بيحجب **كل** أعمدة الكيان المرتبط — وده في مشروعنا ممكن يجيب أعمدة `varbinary` فيها صور، وده تحميل ثقيل بلا أي داعي.

6.4. Translation Isn't Always Possible

لو كتبت في الـ `Where` دالة `C#` مبيعرفش يترجمها، EF Core إما يرمي استثناء أو (في إصدارات قديمة) يجيب الصفوف كلها ويفلتر في الذاكرة. اعرف إن الشرط لازم يكون قابل للترجمة لـ SQL.

اختبر نفسك

1. امتى الاستعلام ببيروج لقاعدة البيانات فعلًا؟
لما تطلب النتيجة — `ToListAsync` أو `FirstOrDefaultAsync` وخلافه. قبل كده هو مجرد وصف.
2. امتى تستخدم `AsNoTracking()`؟
لما تقرا للعرض بس من غير نية تعديل — بيوفر ذاكرة وشغل تتبّع.
3. إيه هي مشكلة N+1 وإزاي تحلها؟
استعلام لكل عنصر في حلقة. الحل: `Include` أو إسقاط بـ `Select` على الأعمدة المطلوبة بس.
4. ليه الإسقاط أحسن من `Include` في شاشة عرض؟
لأن `Include` بيحجب كل أعمدة الكيان، ويمكن يحجب أعمدة صور ثقيلة من غير داعي.

7. Inheritance: Three User Types, One Table

7.1. What Actually Happens

في الكود عندك ثلاث كلاسات: `Investor` و `Innovator` و `Admin`، وكلهم بيورثوا من `User`. في قاعدة البيانات فيه جدول واحد اسمه `Users`.

```
1 modelBuilder.Entity<User>()
2     .HasDiscriminator<string>("UserType")
3     .HasValue<Investor>("Investor")
4     .HasValue<Innovator>("Innovator")
5     .HasValue<Admin>("Admin");
```

الاستراتيجية دي اسمها **TPH** اختصار Table-Per-Hierarchy: كل الهرم في جدول واحد، وعمود إضافي اسمه `UserType` بيقول الصف ده نوعه إيه.

فين ده في الكود إعداد الـ discriminator — جوه `OnModelCreating`، دور على `HasDiscriminator`
 الكلاس الأب
 النوع الوارث — المستثمر
 النوع الوارث — المؤسس

`Data/AppDbContext.cs`
`Data/Models/User.cs`
`Data/Models/Investor.cs`
`Data/Models/Innovator.cs`

اعرف ده كويس

عمود `UserType` مش عمود عادي إنت بتكتب فيه — ده EF Core `discriminator` بيملاه لوحده لما تعمل `new Investor()`، ويستخدمه لوحده لما تعمل `_context.Investors.ToList()` فيضيف `WHERE UserType = 'Investor'` تلقائيًا.

7.2. Why TPH, Not a Table per Type

الاستراتيجية	الميزة	التكلفة
TPH (اللي احنا مستخدمينه)	استعلام واحد من غير <code>JOIN</code> . المصادقة والرسائل والمتابعة بتشتغل على <code>User</code> من غير ما تعرف النوع.	أعمدة الأنواع المختلفة لازم تكون nullable في الجدول المشترك.

الاستراتيجية	الميزة	التكلفة
TPT (جدول لكل نوع)	كل عمود في مكانه الصح، مفيش <code>NULL</code> كل قراءة لمستخدم = <code>JOIN</code> . والمصادقة زيادة.	كل قراءة لمستخدم = <code>JOIN</code> . والمصادقة بتقرا مستخدم في كل طلب.

جدول 12 المقارنة اللي على أساسها اتأخذ القرار.

خد بالك

التكلفة الحقيقية لـ TPH إن أعمدة زي `PreferredIndustries` و `TicketMin` و `TicketMax` (بتاعة `Investor` بس) موجودة في نفس الجدول وبتبقى `NULL` عند المؤسس والأدمن. ده مقصود ومقبول، مش إهمال — ولو اتسألت عنه قول كده بالظبط بدل ما تحاول تبرره.

اختبر نفسك

1. `Admin` مالوش `DbSet`. إزاي تجيبه؟
`_context.Users.OfType<Admin>()` — أو من خلال الـ discriminator مباشرة.
2. ليه اخترنا TPH مش TPT؟
لأن كل طلب مصادق عليه بيقرأ المستخدم، و TPT كان هيجوّل كل قراءة دي لـ `JOIN` بلا داعي.
3. `TicketMin` بتبقى إيه في صف مؤسس؟
`NULL` — وده الثمن المقبول لـ TPH.

8. Soft Delete: The Row Stays, It Just Disappears

8.1. The Problem

قاعدة رقم ٩ في `AGENTS.md` بتقول: ممنوع تمسح صف علاقة عند الرفض — غير `Status` و `Stage` لـ `Declined`. والسبب مش تقني، السبب إن بيانات الطرفين مشتركة: التزام مستثمر وسجل مؤسس بمين دعمه — دول حقائق مشتركة، وخروج طرف مينفعش يمسح نسخة الطرف الثاني.

8.2. The Fix: A Column Plus a Global Filter

```
1 modelBuilder.Entity<User>().HasQueryFilter(u => !u.IsDeleted);
2 modelBuilder.Entity<Project>().HasQueryFilter(p => !p.IsDeleted);
```

السطرين دول من أخطر سطرين في المشروع كله. معناهم إن أي استعلام على `Users` أو `Projects` في أي مكان في التطبيق بيضاف له `WHERE IsDeleted = 0` تلقائيًا، من غير ما اللي كاتب الاستعلام يفتكر.

فين ده في الكود

السطرين، نفسهم — جوه `OnModelCreating`، دور على `HasQueryFilter` والخاصية `IsDeleted` و `DeletedAtUtc` الخاصة `IsDeleted` الخاصة `IsDeleted`

`Data/AppDbContext.cs`
`Data/Models/User.cs`
`Data/Models/Project.cs`

اعرف ده كويس

الفلتر العام + غياب طبقة repository = تركيبة قوية. لأن كل المتحكمات بتكلم `AppDbContext` مباشرة، مفيش طريق يلتف حوالين الفلتر بالغلط. لو كان فيه repositories، كان ممكن حد يكتب استعلام خام يفوت الفلتر.

خد بالك

جدولين بس عندهم فلتر عام: `User` و `Project`. باقي الجداول لأ. فلو اتسألت «هل كل حاجة عندكم soft delete؟» الإجابة الأمنية: لأ — الحذف الناعم على المستخدم والمشروع، والباقي بيتحمي بسلوك الحذف (`Restrict` / `NoAction`) مش بفلتر.

8.3. How to Bypass the Filter When You Need To

الأدمن أحيانًا محتاج يشوف المحذوف:

```
1 _context.Users.IgnoreQueryFilters().Where(u => u.IsDeleted)
```

اختبر نفسك

1. كام جدول عنده global query filter؟ وإيه هما؟

اتنين بس: `User` و `Project`، والاتنين على `!IsDeleted`.

2. ليه مبنمسحش صف العلاقة عند الرفض؟

لأن البيانات مشتركة بين طرفين — قاعدة ٩ في `AGENTS.md`. بنغيّر `Status` و `Stage` لـ `Declined`.

3. إزاي الأدمن يشوف حساب متمسوح؟

`IgnoreQueryFilters()` على الاستعلام.

9. Indexes: Speed, and Something More Important Than Speed

الفهرس في المشروع ده بيعمل حاجتين مختلفتين تمامًا. الأولى معروفة: يسرّع الاستعلام. الثانية أهم بكثير في مشروعنا: يمنع حالة مستحيلة من إنها تحصل أصلًا.

9.1. Type One: Performance Indexes

الفهرس	بيخدم إيه
<code>Project(ModerationStatus, LifecycleStatus, CreatedDate)</code>	استعلام التصفّح العام: هات المشاريع المعتمدة والنشطة مرتّبة بالأحدث. ده أكثر استعمال بيتنفّذ في المنتج.
<code>Project(Stage)</code>	الفلتر بالمرحلة في صفحة الاستكشاف.
<code>Investment(ProjectId, Status)</code>	حساب أرقام التمويل لمشروع.
<code>Investment(InvestorId, Stage)</code>	لوحة المستثمر: علاقاتي وأنا فين فيها.
<code>PaymentTransaction(ProjectId, Status)</code>	جمع المبالغ المموّلة فعليًا لمشروع.
<code>SecurityLog(UserId, CreatedAtUtc)</code>	السجلّ الأمني لمستخدم مرتّب زمنيًا.
<code>Message(ProjectId, SentAt)</code>	محادثات مربوطة بمشروع معيّن.
<code>SavedSearch(UserId, Scope)</code>	عمليات البحث المحفوظة للمستخدم.

جدول 13 فهرس بتخدم استعلامات حقيقية موجودة في المنتج.

9.2. Type Two: Indexes That Enforce Rules

دي اللي لازم تحفظها كويس. الفكرة اسمها **filtered unique index** — فهرس فريد بشرط يعني: «العمود ده لازم يكون فريد، بس في الصفوف اللي بتحقق الشرط ده».

اعرف ده كويس

ليه ده أقوى من فحص في الكود؟ لأن الكود ممكن يخسر السباق. لو اتنين ضغطوا «ادفع» في نفس الجزء من الثانية، الاتنين ممكن يقرأوا «مغيش دفعة ناجحة» في نفس اللحظة وبعدين الاتنين يكتبوا. الفهرس بيتنقذ جوه المحرك نفسه — الصف الثاني بترفض، مغيش سباق.

الكود نفسه مرقم القيود الأربعة في `ConfigureFunding`:

فين ده في الكود

الميثود `ConfigureFunding` — كل القيود الأربعة جواها، وكل واحد فوقه تعليق برقمه
ابحث عن `HasFilter` عشان تلاقي كل الفهارس المفطرة
`Data/AppDbContext.cs`
`Data/AppDbContext.cs`

القيود	بيمنع إيه
<code>UX_FundingRequests_OneOpenPerInvestment</code>	طلبين تمويل مفتوحين لنفس العلاقة — اللي كان هيجلي نفس الصفقة تتمول مرتين.
<code>UX_PaymentTransactions_OneActivePerRequest</code>	محاولتين دفع شغالين في نفس الوقت لنفس الطلب.
<code>UX_PaymentTransactions_OneSucceededPerRequest</code>	الأهم. دفعتين ناجحتين لنفس الطلب. حتى لو كل الحماية الثانية فشلت، قاعدة البيانات نفسها بترفض تسجل نفس التمويل مرتين.
<code>UX_PaymentEvents_ProviderEventId</code>	نفس الحدث من المزود يتطبق مرتين — وبهوك متكرر، أو وبهوك سابق رجوع المستخدم من صفحة الدفع.

جدول 14 القيود الأربعة اللي شايطة سلامة النظام المالي.

وفيه اتنين كمان بره مجموعة التمويل:

- `UX_Investments_OneLivePerInvestor` — بشرط `[InvestorId] IS NOT NULL AND [Status] IN ('Pending', 'Approved')`: مستثمر واحد مايقدرش يكون عنده علاقتين حيتين مع نفس المشروع.
- `UX_TermSheets_OneLivePerInvestment` — بشرط `[Status] = 'Proposed'`: ورقة شروط واحدة معروضة في المرة. اتنين معروضين = اتفاقين كل واحد بيدعي إنه الصفقة.

مثال

إزاي تقرا فهرس مفلتر

```
1 modelBuilder.Entity<PaymentTransaction>()
2   .HasIndex(t => t.FundingRequestId,
3   "UX_PaymentTransactions_OneSucceededPerRequest")
4   .IsUnique()
5   .HasFilter($"[Status] = '{PaymentStatus.Succeeded}'");
```

اقرها كده: «على جدول `PaymentTransactions`، خلي `FundingRequestId` فريد — بس بُص على الصفوف اللي حالتها `Succeeded` بس.»

يعني ممكن يبقى فيه عشر محاولات فاشلة لنفس الطلب (عادي، ومطلوب)، لكن **ناجحة واحدة بالظبط**.

خد بالك

في الكود فيه فهرسين على نفس العمود `FundingRequestId` بأسماء صريحة. الاسم الصريح ده **مش رفاهية**: EF Core بيميز الفهارس بقائمة الأعمدة، فلو الاتنين من غير اسم كانوا هيندمجوا في واحد والتاني يضيع بالصمت — وده كان هيضيع قيد الفلوس بيعتمد عليه. ده تعليق مكتوب حرفياً في الكود، ونقطة ممتازة تقولها.

اختبر نفسك

- إيه الفرق بين فهرس فريد عادي وفهرس فريد مفلتر؟
العادي بيطبق التفرّد على كل الصفوف. المفلتر بيطبقه على الصفوف اللي بتحقق شرط `HasFilter` بس.
- ليه القيد على الدفع في قاعدة البيانات مش في الكود؟
لأن الكود ممكن يخسر سباق (race condition) لو طلبين وصلوا في نفس اللحظة. القيد في المحرك نفسه مبيخسرش.
- ينفع يبقى فيه ه محاولات دفع فاشلة لنفس الطلب؟
أيوه، وده مطلوب. القيد على الحالة `Succeeded` بس — الفاشلة مش داخله في الفلتر.
- ليه الفهرسين على `FundingRequestId` ليهم أسماء صريحة؟
لأن EF Core بيميز الفهارس بأعمدها؛ من غير أسماء كانوا هيندمجوا وواحد يضيع بالصمت.

10. Two Rules That Never Break

في `AGENTS.md` قاعدتين اسمهم عليك إنت مباشرة. اعرفهم بالنص.

10.1. Rule One: Every Date Is UTC

الوقت بيتخزن UTC دائماً، والتحويل للتوقيت المحلي بيحصل في الواجهة بس. السبب: المستخدمين ممكن يكونوا في مناطق زمنية مختلفة، ولو خزّنا التوقيت المحلي، مقارنة تاريخين بتبقى بلا معنى.

الأسماء نفسها بتقول ده: `CreatedAtUtc`, `SucceededAtUtc`, `OnboardedAtUtc`, `DeletedAtUtc`. اللاحقة `Utc` جزء من الاسم عشان محدش يلخبط.

10.2. Rule Two: Every Money Column Is `HasPrecision(18, 2)`

```
1 modelBuilder.Entity<Investment>()
2     .Property(i => i.Amount)
3     .HasPrecision(18, 2)
4     .IsRequired();
```

خد بالك

لو نسيت السطر ده، SQL Server بيرجع لـ `decimal(18,0)` — يعني **يقصّ الكسور بالصمت**. مبلغ ٢٥٠٠,٧٥ بيتخزن ٢٥٠٠. مفيش خطأ، مفيش تحذير، الفلوس بس بتختفي. ده مكتوب حرفياً كتعليق في `AppDbContext`.

فيين ده في الكود

Data/AppDbContext.cs
Data/AppDbContext.cs
Services/UtcDateTimeConverter.cs

كل استدعاءات `HasPrecision` — إبحث عن الكلمة تلاقهم كلهم
الميثود `ConfigureFunding` — دقة أعمدة التمويل والدفع
آلية فرض الـ UTC

الأعمدة اللي عليها دقة صريحة:

العمود	الدقة	ملاحظة
Investment.Amount	18, 2	المبلغ الملتزم به.
Project.InvestmentNeeded	18, 2	هدف الجولة.
Project.Valuation	18, 2	التقييم.
Project.EquityOffered	5, 2	نسبة مئوية — مش محتاجة ١٨ خانة.
TicketMax / Investor.TicketMin	18, 2	نطاق تذكرة المستثمر.
Valuation / TermSheet.Amount	18, 2	شروط الصفقة.
TermSheet.EquityPct	7, 4	انتبه: دقة مختلفة عن كل اللي فوق.
CounterAmount / FundingRequest.Amount	18, 2	الطلب والعرض المضاد.
PaymentTransaction.Amount	18, 2	المبلغ.
PaymentTransaction.FeeAmount	18, 2	العمولة.
PaymentTransaction.NetToFounder	18, 2	الصافي للمؤسس.

جدول 15 كل عمود فلوس ودقته.

اعرف ده كويس

`TermSheet.EquityPct` بـ (7,4) مش (18,2) — وده سؤال ممتاز ممكن يتسأل. السبب مكتوب في الكود: نسبة زي ١٢,٣٧٥% لو اتخزنت بخانتين عشريتين هتتقرب لـ ١٢,٣٨% — يعني حصة حد في شركة انغيرت بالصمت. النسبة محتاجة خانات عشرية أكثر من المبلغ، مش أقل.

اختبر نفسك

- إيه اللي بيحصل لو نسيت `HasPrecision` على عمود فلوس؟
SQL Server بيستخدم `decimal(18,0)` ويقصّ الكسور من غير أي خطأ — الفلوس بتضيع بالصمت.
- ليه `EquityPct` دقتها 7,4 مش 18,2؟
عشان التيسب محتاجة خانات عشرية أكثر؛ التقريب لخانتين بيغير حصة حقيقية.
- ليه كل أسماء التواريخ منتهية بـ `Utc`؟
عشان الاسم نفسه يمنع اللبس — كل تاريخ متخزن UTC والتحويل في الواجهة بس.

11. Migrations: The Schema's History

11.1. The Idea

الترحيل (migration) هو ملف بيوصف **التغيير** بين شكل السكيما القديم والجديد. كل ملف فيه دالتين: `Up()` بتطبق التغيير، و `Down()` بتلغيه.

```
1 dotnet ef migrations add AddUserOnboardedAt
```

الأمر ده بيقارن الكلاسات بآخر لقطة (snapshot) محفوظة، وبيولّد الفرق. يعني إنت **مش** بتكتب الترحيل بإيدك — إنت بتغير الكلاس والأداة بتولّد الفرق، وشغلك إنك **تراجع** اللي اتولّد قبل ما يتطبق.

11.2. The Mechanism Inside: Three Things You Must Know

كل ترحيل بيتعامل مع ثلاث حاجات مختلفة، ولو مفهمتهمش هتتلخبط أول مرة يحصل تعارض.

الحاجة	إيه هي ودورها
ملفات الترحيل <code>Migrations/*.cs</code>	الملف اللي فيه <code>Up()</code> و <code>Down()</code> . ده اللي بيتنفذ. بيتحفظ في Git وبيتشارك مع الفريق.
اللقطة <code>AppDbContextModelSnapshot.cs</code>	صورة كاملة للشكل الحالي للموديل. EF بيقارن كلاساتك بيها عشان يعرف الفرق. ملف متولد — متعدّلوش بإيدك .
جدول التاريخ <code>__EFMigrationsHistory</code>	جدول جوه قاعدة البيانات نفسها، فيه أسماء الترحيلات اللي اتطبقت. ده اللي بيخلي EF يعرف إيه اللي ناقص.

جدول 16 ثلاث حاجات، كل واحدة في مكان مختلف.

اعرف ده كويس

الفكرة المهمة: `dotnet ef database update` بيقرأ جدول `__EFMigrationsHistory` في القاعدة، بيشوف إيه اللي اتطبق، ويطبق الناقص بالترتيب. عشان كده **اسم** الترحيل جزء من هويته — لو غيّرت اسم ترحيل اتطبق خلاص، EF هيفتكركه ترحيل جديد ويحاول يطبقه ثاني.

11.3. Why Conflicts Happen

تخيل السيناريو ده:

1. إنت ولدت ترحيل **A**. اللقطة اتحدّثت عندك.
 2. زميلك ولد ترحيل **B** من نفس الأساس القديم. اللقطة اتحدّثت عنده.
 3. الاتنين عملتوا `push`.
- النتيجة: تعارض في `AppDbContextModelSnapshot.cs` (ملف واحد اتعدّل في مكانين)، وكمان ترحيل **B** مبني على أساس مش شايف **A**.

خد بالك

حلّ التعارض في ملف اللقطة يدويًا **فكرة سيئة** — الملف كبير ومتولد. الحل العملي الأسلم: واحد بس في الفريق هو اللي يولد الترحيلات (وده إنت). أي حد عايز تغيير في السكيمات، يقولك وإنت تعمله. ده مش تحكّم — ده تجنّب لمشكلة تكلفتها أكبر بكثير من التنسيق.

11.4. The Safest Command You Have

```
1 dotnet ef migrations script
```

بيطبع الـ SQL اللي هيتنفذ، على الشاشة، من غير ما يلمس القاعدة. استخدمه قبل أي `database update` عشان تشوف بعينك التغيير الحقيقي على مستوى SQL — مش على مستوى `C#`.

11.5. All Twenty-Six Migrations

الجدول ده بالترتيب الزمني الحقيقي. مش مطلوب تحفظه — مطلوب تعرف تقرا منه لو اتسألت «امتى دخل كذا؟».

#	الترحيل	أضاف إليه
١	initialCraete	الأساس: <code>Users</code> , <code>Projects</code> , <code>Investments</code> , <code>Messages</code> , <code>Comments</code> , <code>Replies</code> , <code>Notifications</code> . الاسم فيه غلطة إملائية مقصود إنها تتساب.
٢	Phase4ProductionAuthSecurity	<code>RefreshTokens</code> , <code>SecurityLogs</code> , وأعمدة تأكيد الإيميل والقفل بعد المحاولات الفاشلة.
٣	Phase5SearchAndProjectImages	<code>ProjectImages</code> , وحقول البحث على المشروع: <code>Category</code> , <code>Industry</code> , <code>Location</code> .
٤	AddInvestmentApprovalStatus	عمود <code>Investment.Status</code> — بوابة التمويل.
٥	AddBookmarks	<code>Bookmarks</code> بقيد فريد (<code>UserId</code> , <code>ProjectId</code>).
٦	AddUpdatesAndMilestones	<code>ProjectUpdates</code> , <code>ProjectUpdateImages</code> , <code>Milestones</code> .
٧	AddTeamAndDocuments	<code>TeamMembers</code> , <code>ProjectDocuments</code> .
٨	AddFollowsAndProfileMedia	<code>Follows</code> بقيد فريد، وعمود <code>CoverImage</code> .
٩	AddMessageIsRead	عمود <code>Message.IsRead</code> — أساس عدّاد غير المقروء.
١٠	AddModerationAndAnalytics	<code>ModerationStatus</code> , <code>Reports</code> , <code>ProjectViews</code> , <code>Reviews</code> , <code>AdminAuditLogs</code> .
١١	AddSocialLinksAndInvestorInterests	روابط التواصل، و <code>PreferredIndustries</code> .
١٢	AddTeamMemberEmail	عمود <code>TeamMember.Email</code> .
١٣	AddLastSeenAndMessageAttachments	<code>LastSeenAt</code> , وجدول <code>MessageAttachments</code> .
١٤	FixesPass1_ModerationDecimalContactAudit	مهم: ضبط الدقّة العشرية على أعمدة الفلوس، و <code>ContactInfo</code> , وسجلّ التدقيق، والحذف الناعم.
١٥	Phase2_InvestmentPipeline	عمود <code>Investment.Stage</code> , والملاحظات الخاصة، و <code>DocumentDownloadLogs</code> .
١٦	Phase5_SuspensionAndModerationNotes	<code>IsSuspended</code> , <code>ModerationNote</code> .
١٧	Phase6_VentureLifecycle	<code>LifecycleStatus</code> , وأعمدة إغلاق الجولة.
١٨	BrowseDiscoveryIndexes	فهارس التصفح المركّبة — أداء بحث.
١٩	InvestorProfileThesisAndTicket	<code>InvestmentThesis</code> , <code>TicketMin</code> , <code>TicketMax</code> , <code>ListedInDirectory</code> .
٢٠	NotificationPrefsAndSelfDelete	<code>NotifyOn*</code> , <code>DeletedAtUtc</code> .
٢١	ExpansionRelationshipWorkspace	<code>DealQuestions</code> , <code>DocumentRequests</code> , <code>SavedSearches</code> , والرسائل المرتبطة بمشروع. وكمات شال جدولي <code>Payments</code> و <code>ProjectAnalyses</code> القدام.
٢٢	FundingRequestsAndSandboxPayments	أهم ترحيل مالي: <code>FundingRequests</code> , <code>PaymentTransactions</code> , <code>PaymentEvents</code> + القيود الأربعة.
٢٣	AddUserOnboardedAt	عمود <code>User.OnboardedAtUtc</code> — أصغر ترحيل، وأوضح مثال على التغيير الآمن.

#	الترحيل	أضاف إيه
٢٤	OneLiveInvestmentPerInvestor	القيد UX_Investments_OneLivePerInvestor
٢٥	StageHistoryRemindersAndReconciliation	InvestmentStageEvents
		FundingRequest.RemindersSent ، وأعمدة مراجعة
		PaymentEvent
٢٦	TermSheetsCounterOffersAndTwoWayDocs	TermSheets والعروض المضادة والشرائح، و
		DealQuestion.ParentQuestionId

جدول 17 الستة وعشرون ترحيلًا بالترتيب الزمني.

اعرف ده كويس

ثلاث ترحيلات تستاهل تعرفهم بالاسم لو اتسألت:

- رقم ١٤ — لأنه بيثبت إن الدقة العشرية اتضبطت بعد ما اتنسيت، يعني القاعدة اتطبقت بأثر رجعي.
- رقم ٢٢ — لأنه اللي جاب النظام المالي كله والقيود الأربعة.
- رقم ٢٣ — لأنه أصغر واحد وأوضح مثال على الترحيل الآمن.

خد بالك

اسم الترحيل الأول initialCraete فيه غلطة إملائية (Craete بدل Create). متصلّحهاش. اسم الترحيل جزء من هويته في جدول __EFMigrationsHistory، وتغييره بيكسر التتبع عند كل حد في الفريق. لو اتسألت عنها قول إنها معروفة ومتسابة عن قصد.

12. A Shared Database: The Risk and the Discipline

12.1. The Problem

AGENTS.md يقول بالنص: قاعدة البيانات بعيدة ومشتركة. متشغّلش dotnet ef database update من غير ما تقول. معنى ده إن الأمر اللي بتكتبه إنت على جهازك بيغيّر القاعدة اللي الستة أعضاء شغّالين عليها في نفس اللحظة.

مفيش نسخة محلية. مفيش «أجرّب الأول عندي».

12.2. Checklist Before You Apply

1. وُلِد الترحيل بس، متطبّقش. dotnet ef migrations add <Name>
2. افتح الملف واقراه سطر سطر. شوف Up() بيعمل إيه بالضبط.
3. اسأل: هل ده إضافي بس؟ إضافة عمود nullable أو جدول جديد = آمن. حذف عمود أو تغيير نوع أو إضافة عمود NOT NULL من غير قيمة افتراضية = خطر على بيانات موجودة.
4. اقرا Down(). لو مفيش طريق رجوع نضيف، فكّر ثاني.
5. قول للفريق. قبل، مش بعد.
6. طبّق. dotnet ef database update

12.3. The Real Example: AddUserOnboardedAt

ده ترحيل اتعمل فعلاً في المشروع، وهو أحسن مثال تشرح بيه القائمة اللي فوق.

```
1 protected override void Up(MigrationBuilder migrationBuilder)
2 {
3     migrationBuilder.AddColumn<DateTime>(
4         name: "OnboardedAtUtc",
5         table: "Users",
6         type: "datetime2",
7         nullable: true);
8 }
9
10 protected override void Down(MigrationBuilder migrationBuilder)
11 {
12     migrationBuilder.DropColumn(
13         name: "OnboardedAtUtc",
14         table: "Users");
15 }
```

مثال

ليه اتحكم عليه إنه آمن

- **عمود واحد بس**، ومفيش أي تغيير ثاني اتسرب في الترحيل (ده اتراجع فعلياً قبل التطبيق).
- **nullable**: الصفوف الموجودة كلها بتأخذ **NULL** من غير ما حد يلمسها، ومن غير قيمة افتراضية مخترعة.
- **ميكسersh الكود الشغال**: أي نسخة قديمة من التطبيق شغالة عند زميلك مش بتعرف العمود ده أصلاً، فمش هتتأثر.
- **Down()** **نضيغة**: بيثيل العمود، خلاص. الرجوع ممكن.

فين ده في الكود

Migrations/20260809201317_AddUserOnboardedAt.cs
Data/Models/User.cs
Migrations/AppDbContextModelSnapshot.cs

الترحيل نفسه — **Up()** و **Down()** والتعليق اللي فوقها الخاصة **OnboardedAtUtc** بيقرن بيها — متعدّلهاش بإيدك اللقطة اللي EF

اعرف ده كويس

الفرق بين عمود **nullable** وعمود **NOT NULL** مع قيمة افتراضية مش تفصيلة أسلوب. **nullable** معناه «الحقيقة دي مش معروفة لصفوف قديمة» — وده الصح هنا، لأن المستخدمين القدام فعلاً **مش معروف** هما خلّصوا الأونبوردنج امتى. **NOT NULL DEFAULT GETUTCDATE()** كان هيكذب: كان هيقول إن كل المستخدمين القدام خلّصوا الأونبوردنج لحظة الترحيل.

13. File Storage: Why the Bytes Live in the Database

ده قرار في السكيميا، فهو شغلك، وهو كمان **مقايضة معروفة وموتّقة** — يعني سؤال متوقّع جداً.

13.1. The Current Setup

كل الصور والمستندات في المشروع متخزنة كأعمدة **varbinary(max)** جوّه قاعدة البيانات نفسها:

المكان	إيه اللي متخزن
User.ProfileImage	صورة الحساب.
User.CoverImage	صورة الغلاف.
ProjectImage.ImageData	صور المشروع.
ProjectUpdateImage	صور تحديثات المشروع.
ProjectDocument	مستندات غرفة البيانات.
MessageAttachment.Data	مرفقات الرسائل.

جدول 18 كل مكان فيه بايتس ملفات في السكيميا.

13.2. The Alternative That Wasn't Used

الطريقة الاحترافية في الإنتاج: تخزن الملف في خدمة تخزين خارجية (S3 أو Azure Blob) وتحفظ في القاعدة **الرابط بس**.

تخزين خارجي	بايتس في القاعدة (وضعا)
بيفضل صغير	بيكبر بسرعة
خفيف	بطيء وثقيل
بيتسحب من الخدمة مباشرة	الملف بيتحمل في الذاكرة
محتاج حساب وإعداد ومفاتيح	صفر — مفيش خدمة ثانية
ممکن يحصل صف بلا ملف	الملف والصف بيتحفظوا في معاملة واحدة

جدول 19 المقايضة بالتفصيل.

اعرف ده كويس

الإجابة الأمينة لو اتسألت: «إحنا عارفين إن ده مش الاختيار الصح للإنتاج. اخترناه لأن التخزين الخارجي بره نطاق المشروع — كان هيجتاج حساب سحابي ومفاتيح وإدارة، والمقابل مش مستحق في مشروع تخزج. والمكسب إن الملف والصف بيتحفظوا في معاملة واحدة، فمستحيل يبقى فيه صف مستند بلا ملف.»
ودي مقايضة **مكتوبة** في `docs/09-STATUS.md` كحدّ معروف — مش حاجة اكتشفتها اللجنة.

خد بالك

متحاولش تدافع عن `varbinary` كانه الاختيار الأفضل تقنيًا — مش هو. الدفاع الصح إنه اختيار **مناسب للنطاق** مع معرفة كاملة بتكلفته. الفرق بين الإجابتين هو الفرق بين طالب فاهم وطالب بيبرر.

14. Known Limitations in Your Area

الفصل ده أهم من أي فصل تاني في لحظة واحدة: لما اللجنة تسأل «إيه العيوب؟». اللي بيعرف عيوب شغله بيبان إنه فاهمه.

الحدّ	الوضع والتبرير
قاعدة واحدة مشتركة	مفیش نسخة محلّية لكل مطوّر. أي ترحيل بيمسّ الفريق كله فوراً. الانضباط التنظيمي هو الحماية الوحيدة.
الملفات جوّه القاعدة	<code>varbinary(max)</code> — مقايضة مقصودة، الفصل اللي فات.
مفیش اختبارات على القاعدة	ال ٤٨ اختبار كلهم على منطق الحسابات والمراحل، ومبيلمسوش قاعدة بيانات. القيود والفهارس اللي كتبها مش مغطّاة باختبار — فجوة موثّقة في <code>docs/09-STATUS.md</code> .
الحذف الناعم جزئي	على <code>User</code> و <code>Project</code> بس. الباقي محمي بسلوك الحذف مش بفلتر.
مفیش صلاحيات دقيقة	الأدوار خشنة (<code>Admin</code> / <code>Innovator</code> / <code>Investor</code>) وبس.

جدول 20 حدود حقيقية، كل واحد وسببه.

اعرف ده كويس

لو اتسألت «إيه أهم حاجة ناقصة في شغلنا؟» أقوى إجابة أمينة: **اختبارات integration على القيود المالية**. القيود الأربعة اللي شايلة سلامة الفلوس مبنية صح، لكن مفیش اختبار بيحاول يكسرها ويتأكد إنها بترفض. ده أول حاجة كنت هعملها لو فيه وقت إضافي.

15. Explaining the ERD in 5 Minutes

ده سيناريو كلام تقوله بصوتك قدام الدكتور وإنت فاتح الرسمة. مقسّم بالوقت. كل اللي فيه اتشرح في الفصول اللي فاتت — ده ترتيبه للكلام مش معلومات جديدة.

اعرف ده كويس

القاعدة الذهبية: **ابدأ بالمركز، مش بالتفاصيل**. أسوأ حاجة تعملها إنك تبدأ تعدّ الجداول واحد واحد. ابدأ بالكيانات الثلاثة اللي المنتج قايم عليهم، وبعدين وسّع.

15.1. Minute One: The Big Picture

«المخطط قايم على ثلاث كيانات أساسية:

User — المستخدم. وده مقسّم لتلات أنواع: مستثمر ومؤسّس وأدمن، بس كلهم في **جدول واحد** بعمود اسمه `UserType` بيميّز بينهم. الاستراتيجية دي اسمها TPH.

Project — المشروع اللي المؤسّس بينشره.

Investment — العلاقة بين مستثمر ومشروع. ودي **مش جدول ربط عادي** — دي كيان كامل ليه حالة ومرحلة وتاريخ وملاحظات.»

15.2. Minute Two: Why `Investment` Isn't a Join Table

«لو كانت مجرد ربط، كانت هتبقى عمودين وخلص. لكن عندها:

- `Status` — بوابة الفلوس: `Declined`, `Approved`, `Pending`.
- `Stage` — مرحلة العلاقة بين الطرفين، وعندنا تمان مراحل.

والاثنين دول **محورين مستقلين**: علاقة ممكن تكون **Approved** — يعني الفلوس محسوبة — وفي نفس الوقت **InDiscussion** يعني لسه بيتكلموا.»

خد بالك

لو الدكتور قاطعك هنا وسأل «ليه مدمجتوهمش؟» — الإجابة جاهزة: «لأنهم بيقيسوا حاجتين مختلفتين. الدمج كان هيخلي مستحيل تعرف رقم التمويل من غير ما تقرا سياق المحادثة.»

15.3. Minute Three: The Money Chain

لما الطرفين يتفقوا، بتبدأ سلسلة من أربع خطوات:

Investment ← **TermSheet** ← **FundingRequest** ← **PaymentTransaction** ← **PaymentEvent**

- **TermSheet** — الشروط المكتوبة اللي الطرفان قبلوها.
- **FundingRequest** — طلب المؤسس للفلوس المتفق عليها.
- **PaymentTransaction** — كل محاولة دفع، وكل محاولة صف مستقل.
- **PaymentEvent** — كل تأكيد جاي من مزود الدفع.»

15.4. Minute Four: The Part That Matters

أهم حاجة في المخطط ده إن سلامة الفلوس **مش في الكود** — هي في قاعدة البيانات نفسها. فيه أربع قيود فريدة مفطرة:

1. طلب تمويل واحد مفتوح لكل علاقة.
 2. محاولة دفع نشطة واحدة لكل طلب.
 3. **دفعة ناجحة واحدة** لكل طلب — دي الأهم.
 4. حدث واحد لكل معرّف من المزود — دي اللي بتمنع الوبهوك المتكرر.
- والسبب إنها في قاعدة البيانات مش في الكود: الكود ممكن يخسر سباق لو طلبين وصلوا في نفس اللحظة. القيد في المحرّك نفسه مبيخسرش.»

15.5. Minute Five: The Close

«وفيه قاعدتين على مستوى المخطط كله:

- كل تاريخ متخزن UTC — والأسماء نفسها بتقول ده، كلها منتهية ب **Utc**.
- كل عمود فلوس دقته **18,2** — ما عدا نسبة الحصة **EquityPct** دقتها **7,4** لأن التقريب لـ **Declined** كان هيتغير حصة حقيقية.

وأخيراً: إحنا مبمنسحش صفوف العلاقات. لو اترفض طلب، الصف بيفضل والحالة بتتغير لـ **Declined** — لأن البيانات مشتركة بين طرفين ومينفعش خروج طرف يمسح تاريخ الثاني.»

اعرف ده كويس

لو فضل وقت أو الدكتور طلب زيادة، النقطة اللي تفتحها: **الفرق بين Restrict و Cascade و SetNull** — وخصوصاً إن **SecurityLog** هو الوحيد بـ **SetNull** عشان السجل يفضل حتى لو الحساب راح.

فين ده في الكود

Data/AppDbContext.cs
Data/Models/Investment.cs
Services/PipelineStages.cs

كا . العلاقات والقصد — الميثود **ConfigureFunding** و **OnModelCreating** — المحورين **Status** و **Stage** — المراحل الثمانية بأسمائها

16. Expected Questions and Their Answers

الإجابات دي مكتوبة بالطريقة اللي هتقولها بصوتك، مش بصياغة تقرير.
الأسئلة دي معاها **أسئلة الضغط** — يعني المتابعة اللي ممكن تيجي لو الدكتور مقتنعش من أول إجابة. كل الإجابات من المادة اللي فاتت.

مثال

«إشرحلي السكيما بتاعتكم»

ابدأ بالمركز مش بالتفاصيل: «فيه **User** وبيتفرّع منه ثلاث أنواع في نفس الجدول. المؤسس بينشر **Project**. المستثمر بيعمل **Investment** على المشروع — ودي مش مجرد ربط، دي علاقة ليها حالة ومرحلة. لما الاتنين يتفقوا، بيتولّد **FundingRequest**، وكل محاولة دفع بتتسجّل **PaymentTransaction**، وكل تأكيد من المزود بيتسجّل **PaymentEvent** مستقل». وبعدين افتح ال ERD وورّي.

لو ضغط: «طب ليه **Investment مش مجرد جدول ربط؟»** «لأن عندها **Status** و **Stage** وتاريخ وملاحظات خاصة لكل طرف. جدول الربط عمودين وخلاص.»

لو ضغط أكثر: «وليه **PaymentEvent جدول لوحده مش عمود؟»** «عشان الحدث لازم يتسجّل قبل ما أي رقم يتحرّك. لو كان عمود، مكنش فيه مكان أسجّل فيه ويهوك اتكرّر أو ويهوك وصل قبل رجوع المستخدم من صفحة الدفع.»

مثال

«ليه مفيش repository layer؟»

«ده قرار معماري مكتوب في قواعد المشروع: المتحكّمات بتكلّم **AppDbContext** مباشرة. السبب إن ال **DbContext** نفسه أصلًا فيه نمط ال Unit of Work وال Repository جّواه، طبقة زيادة فوقه كانت هتبقى تغليف بلا قيمة في مشروع بالحجم ده. وفيه فائدة إضافية: ال global query filters بتاعتي بتسري على كل استعلام من غير ما حد يقدر يلتفّ حوالها.»

لو ضغط: «طب ده مش بيصعب تغيير قاعدة البيانات بعدين؟» «نظريًا أيوه. عمليًا ال **DbContext** نفسه هو طبقة التجريد — لو غيرنا المحرّك، اللي بيتغير هو مزود EF Core مش كود المتحكّمات.»

لو ضغط أكثر: «إزاي بتختبروا المتحكّمات وهي مربوطة بالقاعدة؟» «الأمانة أقوى: «مبتخبرهاش. ال ٤٨ اختبار الموجودين على منطق الحسابات والمراحل بس، ومفيش اختبارات integration — ودي فجوة موثقة عندنا.»

مثال

«ليه الشكل ده بالذات للسكيما؟»

دي مش بتاعتك — سلّمها بنضافة: «القرار المعماري ده بتاع قائد الفريق، هو صاحب قصة ليه اتصمّت كده. أنا مسؤول عن إنها مبنية صح: العلاقات، الفهارس، القيود، والترحيلات.» ده مش تهرب — ده توزيع أدوار واضح، واللجنة بتحترمه أكثر من إجابة مخترعة.

لو ضغط: «طب إنت عارف ليه ولا بتنقل بس؟» متدعّيش ملكية القرار، بس وّرّي إنك فاهمه: «فاهم المنطق — إن العلاقة كيان مستقل عشان تشيل حالة ومرحلة، وإن الأحداث بتتفصل عن المعاملات عشان منع التكرار. بس القرار نفسه واتخذه بتاع قائد الفريق.»

مثال

«ليه الفلوس `decimal` مش `float`؟»
 «لأن `float` بيمثل الأرقام تقريبياً في النظام الثنائي، فكسور زي ٠,١ مبتخزنش بالظبط مع الفلوس ده معناه فروق بتتراكم. `decimal(18,2)` بيخزن القيمة بالظبط.»

لو ضغط: «وليه `18,2` تحديدًا؟» «١٨ خانة إجمالية، منها ٢ عشرية — يعني قروش بالظبط ومساحة كبيرة للمبالغ.»

لو ضغط أكثر: «طب ليه `EquityPct` مختلفة؟» «لأنها نسبة مش مبلغ. دقّتها `7,4` عشان نسبة زي ١٢,٣٧٥% لو اتقرّبت لخانتين كانت هتبقى ١٢,٣٨% — يعني حصّة حد في شركة اتغيّرت.»

مثال

«لو اتنين ولّدوا ترحيل في نفس الوقت؟»
 «هيحصل تعارض في ملف الـ `snapshot`، وأول واحد يطبق بيخلي ترحيل الثاني مبني على أساس قديم. الحل التنظيمي: حد واحد بس بيولّد الترحيلات — وده أنا. اللي عايز تغيير في السكيما بيقولي.»

لو ضغط: «وليه مجلّوش التعارض في ملف اللقطة يدويًا؟» «لأنه ملف متولّد وكبير، وتعديله بإيدك بيخلي الموديل والقاعدة يخلّفوا من غير ما حد يلاحظ التنسيق أرخص من ده بكتير.»

لو ضغط أكثر: «إزاي EF أصلاً عارف إيه اتطبق وإيه لأ؟» «من جدول اسمه `__EFMigrationsHistory` جوّه قاعدة البيانات نفسها، فيه أسماء الترحيلات اللي اتطبقت. عشان كده اسم الترحيل جزء من هويته وممنوع يتغير بعد التطبيق.»

مثال

«إزاي بتضمنوا إن الدافع مبيتسجلش مرتين؟»
 «مش بشرط في الكود — بقيد في قاعدة البيانات. فيه فهرس فريد مفلتر اسمه `UX_PaymentTransactions_OneSucceededPerRequest` على `FundingRequestId` بشرط `Status = 'Succeeded'`. يعني ممكن محاولات فاشلة كتير، لكن ناجحة واحدة بالظبط الكود ممكن يخسر سباق، الفهرس لأ.»

لو ضغط: «طب لو التطبيق حاول يكتب الثانية، هيحصل إيه؟» «قاعدة البيانات بترفض العملية ويرجع استثناء. يعني الفشل بيحصل قبل ما أي رقم تمويل يتحرّك، مش بعده.»

لو ضغط أكثر: «إيه اللي بيمنع نفس الويبهاك يتطبق مرتين أصلاً؟» «قيد تاني: فهرس فريد على `(Provider, ProviderEventId)` في `PaymentEvent`. الحدث بيتسجل الأول، ولو وصل تاني بنفس المعرف بيترفض عند التسجيل.»

لو ضغط أكثر: «والتعديلات المتزامنة على نفس الصف؟» «دي حماية تالّة مختلفة: عمود `RowVersion` على `PaymentTransaction`. لو اتنين قروا نفس الصف وحاولوا يعدّلوه، أول واحد بيكسب والتاني بياخد `DbUpdateConcurrencyException`.»

مثال

«إيه أكثر حاجة ممكن تغلط فيها في شغلك؟»
 سؤال أمانة، وأحسن إجابة أمانة: «ترحيل غير قابل للرجوع على قاعدة مشتركة. عشان كده بقرا `Up()` و `Down()` قبل ما أطبق، وبتأكد إن التغيير إضافي، وبقول للفريق قبل ما أشغل الأمر.»

مثال

«ليه استخدمتوا TPH مش جدول لكل نوع؟»
 «لأن كل طلب مصادق عليه بيقرأ المستخدم. لو كل نوع في جدول لوحده، كل قراءة دي كانت هتبقى JOIN. والتكلفة اللي قبلناها إن أعمدة المستثمر — زي TicketMin و PreferredIndustries — بتبقى NULL عند المؤسس والأدمن.»

مثال

«إيه الفرق بين `Status` و `Stage`؟»
 سؤال شبه مضمون. «Status» بوابة الفلوس: Approved معناها الالتزام بقى محسوب في أرقام التمويل. Stage بيوصف العلاقة بين شخصين: New, Reviewing, Contacted, InDiscussion. وهكذا. علاقة ممكن تكون Approved و InDiscussion في نفس الوقت — الفلوس محسوبة وهما لسه بيتكلموا. دمج العمودين كان هيخلي مستحيل تعرف رقم التمويل من غير ما تقرا سياق المحادثة.»

مثال

«الملفات جوّة قاعدة البيانات؟ ليه؟»
 ماندافعش عنه كأفضل حل. «أيوه، varbinary(max). عارفين إن الصح في الإنتاج تخزين خارجي والقاعدة تحفظ الرابط بس. اخترنا كده لأن الخدمة الخارجية بره نطاق المشروع، والمكسب إن الملف والصف بيتحفظوا في معاملة واحدة. التكلفة: القاعدة بتكبر والنسخ الاحتياطي بيتقل — ودي مقايضة مكتوبة عندنا في docs/09-STATUS.md.»

مثال

«القيود دي مختبرة؟»
 الأمانة أقوى من التبرير. «لأ. ال ٤٨ اختبار الموجودين على منطق الحسابات والمراحل ومبيلمسوش قاعدة بيانات. القيود الأربعة مبنية صح ومراجعة، لكن مفيش اختبار بيجاول يكسرهما. دي أول حاجة كنت هضيفها لو فيه وقت.»

مثال

«لو عايز تصنيف حقل جديد لكيان، اعمل إيه بالترتيب؟»
 «أضيف ال property في الكلاس. لو محتاج إعداد خاص — دقة أو فهرس — أضيفه في OnModelCreating. أعمل dotnet ef migrations add باسم وصفي. أفتح الملف الناتج وأقرا Up() و Down(). أشغل migrations script أشوف ال SQL. أتأكد إن التغيير إضافي. أقول للفريق. وبعدين database update.»

17. Suggested Study Plan

الملف ده كبير، فده ترتيب عملي تمشي بيه.

الجلسة	تقرا إيه	تعمل إيه على الجهاز
الأولى	الفصول ١ و ٢ — دورك وحدودك، وإزاي EF Core بيتشغل	dotnet tool restore ثم dotnet build. افتح ApplicationDbContext.cs وبصّ عليه من غير ما تحاول تفهم كل حاجة.
الثانية	الفصول ٣ و ٤ — العلاقات وتفاصيل الأعمدة	افتح ال ERD جانب الفصل. اتبع كيان واحد (Investment) من الكلاس للـ OnModelCreating للترحيل.

الجلسة	تقرا إيه	تعمل إيه على الجهاز
الثالثة	الفصول ٥ و ٦ و ٧ — التزامن والاستعلامات والوراثة	دور على <code>RowVersion</code> في الكود وشوف إزاي التزامن بيتفحص، وجرب استعلام LINQ وشوف الـ SQL الناتج منه.
الرابعة	الفصول ٨ و ٩ — الحذف الناعم والفهارس	دور في الكود على القيود الأربعة بأسمائها واقراهم في <code>ConfigureFunding</code> .
الخامسة	الفصول ١٠ و ١١ و ١٢ — القاعدتان والترحيلات والقاعدة المشتركة	افتح <code>AddUserOnboardedAt.cs</code> — أصغر ترحيل وأوضح مثال.
السادسة	الفصول ١٣ و ١٤ و ١٥ و ١٦ — التخزين والحدود والـ ERD والأسئلة المتوقعة	جاوب على كل «اختبر نفسك» في الملف من غير ما تبص على الإجابة.
الأخيرة	الفصل الأخير — المراجعة السريعة	اشرح السكيمات بصوت عالي لحد ثاني في ٥ دقائق.

جدول 21 سبع جلسات، كل واحدة قراءة + حاجة بإيدك.

اعرف ده كويس

أهم نصيحة في الملف ده: **متذاكرش بالقراءة بس**. افتح الكود جانب كل فصل. اللجنة مش بتسأل «إيه تعريف الفهرس المفلتر» — بتسأل «وريني فين» و«ليه». واللاتين دول محتاجين إنك تكون فتحت الملف بإيدك قبل كده.

18. Quick Review Before the Defense

خلاصة

ملفك الأساسي: `Data/AppDbContext.cs` — الموديل كله متعرف فيه.

الاستراتيجية: Code-First. الكلاس أصل، الجدول نتيجة، الترحيل هو الجسر.

الوراثة: TPH — ثلاث أنواع مستخدمين في جدول `Users` بعمود `UserId`.

الحذف الناعم: فلتير عام على `User` و `Project` بس. الباقي بيتحمي بسلوك الحذف.

أخطر أربع قيود: طلب مفتوح واحد، محاولة نشطة واحدة، دفعة ناجحة واحدة، وحدث مزود واحد. كلهم فهارس فريدة مفلتر، مش شروط في الكود.

قاعدتان: كل تاريخ UTC، وكل عمود فلوس `HasPrecision(18,2)` — ما عدا `EquityPct` بـ `(7,4)`.

الترحيلات: ٢٦، وآخر واحد `AddUserOnboardedAt`. القاعدة مشتركة — راجع قبل ما تطبق.

حدك: الميكانيكا بتاعتك. «ليه اتصممت كده» بتاعة القائد.

18.1. The Numbers to Memorize

الرقم	إيه هو
٢٦	عدد الترحيلات.
٤٨	عدد الاختبارات — كلها على المنطق، ولا واحد على قاعدة البيانات.
٤	القيود اللي شايلة سلامة الفلوس.
٢	عدد الجداول اللي عليها فلتير حذف ناعم: <code>User</code> و <code>Project</code> .

الرقم	إيه هو
٨	عدد مراحل Stage ، منهم اثنين نهائيين: Closed و Declined .
18,2	دقة كل عمود فلوس.
7,4	دقة EquityPct وحدها.

جدول 22 أرقام تتقال بثقة.

18.2. Where to Find Everything

Data/AppDbContext.cs
 Data/AppDbContext.cs - `ConfigureFunding`
 Data/Models/
 Migrations/
 Services/UtcDateTimeConverter.cs
 Services/PipelineStages.cs

فين ده في الكود
 الموديل كله: العلاقات، الفهارس، الفلاتر، الدقة
 القود الأربعة
 كلاسات الكيانات كلها
 الـ ٣٦ ترحيلًا + اللقطة
 فرض الـ UTC
 المراحل الثمانية

18.3. The First Sentence You Say If Asked to Explain

«المخطط قايم على ثلاث كيانات: **User** وبيتفرّع لثلاث أنواع في جدول واحد، و **Project** اللي المؤسس بينشره، و **Investment** اللي هي العلاقة بينهم — ودي مش جدول ربط، دي كيان ليه حالة ومرحلة.»
 ولو كمل، امشي على سيناريو الخمس دقائق في فصل Explaining the ERD in 5 Minutes.

18.4. One Last Thing

اعرف ده كويس
 لو مش عارف إجابة: **قول مش عارف، وقول مين يعرف.** «دي بتاعة قائد الفريق» أو «دي عند عضو الباكند» إجابة محترمة. الإجابة المخترعة هي اللي بتوقع.